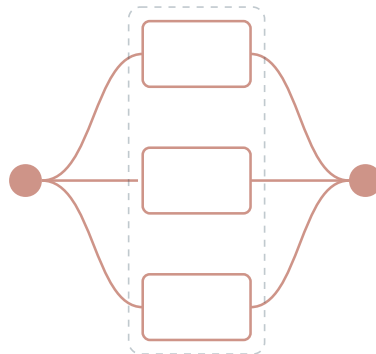


PREMIÈRE ÉDITION · 2026



Le Fil d'Exécution

Concevoir, exploiter et gouverner
des workflows automatisés avec n8n

Ismaël Ladjohounlou

SPÉCIALISTE AUTOMATISATION

Le Fil d'Exécution

Concevoir, exploiter et gouverner des workflows automatisés avec n8n

Ismaël Ladjohounlou

Première édition, 2026

Composé en TeX Gyre Pagella, TeX Gyre Heros et Latin Modern Mono.

Figures originales en SVG. Mise en page A4 produite par WeasyPrint.

Rédigé sur la base de n8n 2.38, avec traitement complet de la migration vers la version 3.0. Les comportements décrits proviennent de la documentation officielle et des notes de version, avec mention de la version d'apparition. Les tarifs et les grilles de prix sont donnés avec leur date de relevé et doivent être revérifiés auprès des éditeurs concernés.

n8n est une marque de n8n GmbH. Cet ouvrage n'est ni édité ni approuvé par n8n GmbH.

Pourquoi j'ai écrit ce livre

Il existe beaucoup de contenu sur n8n et presque aucun ouvrage. Ce qui circule se répartit en deux gisements également insuffisants, et c'est dans l'espace entre les deux que ce livre se place.

D'un côté, des tutoriels de surface qui montrent comment brancher trois nœuds et produire une démonstration. De l'autre, une documentation officielle excellente mais organisée en référence : elle répond parfaitement à la question « comment fonctionne ce nœud », et jamais à la question « comment passe-t-on d'un prototype qui marche sur un poste de développement à un système que l'on ose facturer à un client ».

C'est la seconde question que je traite ici. Je pars du principe que construire est la partie facile, et que la difficulté commence après : quand une API change de format, quand un webhook est livré deux fois, quand un ouvrier ne déchiffre plus les identifiants, quand un dirigeant demande ce que tout cela rapporte et que personne n'a de chiffre.

Deux pistes de lecture

Écrire pour deux publics dans un seul volume échoue presque toujours, parce qu'on finit par écrire pour un lecteur moyen qui n'existe pas. Chaque chapitre indique donc en tête à qui il s'adresse en priorité.

La **piste praticien** couvre les vingt-trois chapitres dans l'ordre. Elle s'adresse aux développeurs, intégrateurs et consultants techniques qui vont construire et exploiter.

La **piste décideur** emprunte les chapitres 1, 2, 4, 7, 10 et 16, puis les six cas d'usage et la clôture. Elle s'adresse à ceux qui doivent arbitrer, budgéter et encadrer. Aucun de ces chapitres ne suppose la lecture d'un chapitre réservé à la piste praticien.

Une instance, plusieurs commanditaires

J'ai construit ce livre sur un fil conducteur unique. Le plan technique est continu : une seule instance le traverse, avec les mêmes conventions et le même dépôt, si bien que l'installation passée en mode queue au chapitre 8 est celle qui fait tourner l'assistant WhatsApp du chapitre 17. Le plan métier, lui, est discontinu : chacun des six cas d'usage arrive avec son commanditaire, ses contraintes et ses chiffres, et s'ouvre par tout ce qu'il faut pour être lu seul.

Une remarque sur les versions

La plateforme évolue vite et par ruptures. La version 2.0 a durci le produit fin 2025 ; la version 3.0 retire des nœuds entiers et impose Docker en auto-hébergement. J'ai écrit sur la base de la 2.38 et consacré le dernier chapitre à la migration, parce qu'un ouvrage qui enseignerait seulement un état de l'outil serait périmé avant d'être lu. Ce que je transmets ici n'est pas une version : c'est la manière de maintenir un parc d'automatisations à travers les ruptures.

TABLE DES MATIÈRES

Vingt-trois chapitres

PARTIE I · FONDATIONS

1	Ce que l'automatisation résout, et ce qu'elle ne résout pas	8
2	Le paysage 2026 : où se situe n8n	13
3	Anatomie de n8n	18

PARTIE II · CONCEVOIR

4	Du processus métier au diagramme de workflow	24
5	Données, expressions et chaînage d'items	28
6	Déclencheurs, webhooks et intégrations	32
7	Erreurs, reprises et idempotence	36

PARTIE III · EXPLOITER

8	Déploiement : du Compose au mode queue	41
9	Sécurité, identités et secrets	46
10	Observabilité et preuve de valeur	50
11	Cycle de vie : versions, environnements, CI/CD	53

PARTIE IV · CONVERSATIONNEL ET IA

12	Canaux conversationnels : chat web et WhatsApp	56
13	De la règle à l'agent	60
14	RAG et connaissance métier	64
15	Orchestration multi-agents	67
16	Fiabilité et gouvernance des systèmes IA	69

PARTIE V · LE SIX-PACK

17	Assistant WhatsApp de premier niveau	72
18	Agent de support documenté	75
19	Réconciliation d'encaissements mobile money	78
20	Devis vers facture depuis canaux mixtes	81

21	Agent d'opérations interne exposé en MCP	84
22	Veille, reporting et preuve du ROI	87

PARTIE VI · CLÔTURE

23	Migrer vers n8n 3.0 et checklist de mise en production	90
A	Conventions de l'instance	95
B	Variables d'environnement de référence	97
C	Grille de choix d'outil	99
D	Glossaire	100
E	Notes et sources	102

Ismaël Ladjohounlou

Je suis ingénieur full-stack et consultant technique indépendant, spécialiste de l'automatisation et de l'intégration de systèmes.

Je conçois, déploie et exploite des chaînes de traitement qui relient les outils métier des organisations : canaux d'entrée, outils de gestion, systèmes de paiement, bases documentaires. J'interviens également sur les infrastructures réseau, ce qui donne à mon travail d'intégration une perspective que n'ont pas toujours les praticiens venus du seul développement applicatif.

Ce qui m'occupe est moins la construction de démonstrations que ce qui vient après : la mise en production, l'idempotence, la reprise après incident, la mesure de la valeur produite et sa défense devant ceux qui financent. C'est cette conviction qui structure le présent ouvrage, et qui explique pourquoi le chapitre le plus important n'est pas celui qui traite des agents mais celui qui traite des erreurs.

J'ai écrit *The Money Path*, consacré à l'intégration sécurisée de Stripe, et je reprends ici la même méthode : partir de la documentation officielle, refuser les chiffres que je ne peux pas expliquer, et donner au lecteur de quoi vérifier plutôt que de quoi croire.

Je travaille depuis Abomey-Calavi, au Bénin, en français et en anglais. Les contraintes de terrain ouest-africaines qui traversent les six cas d'usage de cet ouvrage viennent de missions réelles : réseau intermittent, paiements par monnaie électronique, clients qui écrivent sur trois canaux à la fois. Les patrons qu'on en tire, eux, sont universels.

Je prépare une édition anglaise de cet ouvrage, sous le titre *The Execution Path*, en collection avec *The Money Path*.

Ce que l'automatisation résout, et ce qu'elle ne résout pas

PISTE PRATICIEN ET PISTE DÉCIDEUR

Une automatisation qui échoue le fait rarement pour des raisons techniques. Elle échoue parce qu'on a automatisé un processus que personne dans l'organisation ne savait décrire.

1.1 Le processus que personne ne sait décrire

Demandez à trois personnes d'une même équipe comment une commande est traitée. Vous obtiendrez trois réponses. Non pas parce qu'elles se trompent, mais parce que chacune décrit le chemin qu'elle emprunte, avec ses raccourcis, ses exceptions et les cas particuliers qu'elle a appris à gérer seule. Le processus réel n'existe nulle part sous forme écrite. Il vit dans trois têtes, en trois versions légèrement différentes.

C'est le point de départ de tout projet d'automatisation, et c'est la raison pour laquelle tant de projets s'enlisent au moment de l'implémentation. On ne peut pas coder une règle que personne n'a formulée. Le développeur découvre l'écart au moment où il branche le troisième nœud, pose une question, et déclenche une discussion de quarante minutes entre deux services qui pensaient être d'accord.

Il faut donc inverser l'ordre. Le premier livrable d'une mission d'automatisation n'est pas un workflow, c'est une description écrite du processus, validée par ceux qui l'exécutent. Ce document tient sur une page. Il dit ce qui déclenche le processus, quelles décisions sont prises, sur quels critères, qui tranche en cas de doute, et ce qui se passe quand ça rate. Rédiger cette page prend une demi-journée et clarifie souvent davantage que les trois semaines de développement qui suivent.

Cette page a une seconde vertu, moins avouable et tout aussi utile : elle révèle les processus qu'il ne faut pas automatiser. Quand personne n'arrive à écrire la règle, ce n'est pas que la règle est complexe. C'est qu'il n'y en a pas, et qu'on

LE PREMIER LIVRABLE

Ce n'est pas un workflow. C'est une page A4 décrivant le processus, que tout le monde signe.

prend des décisions au jugé. Un jugement ne s'automatise pas ; il se documente d'abord, se standardise ensuite, et se code éventuellement après.

1.2 Trois questions avant de choisir un outil

Une fois le processus écrit, trois questions décident de son sort. Elles se posent dans cet ordre, et une réponse défavorable à l'une d'elles suffit à arrêter le projet.

FRÉQUENCE

Combien de fois ce processus se déclenche-t-il par mois ? En dessous d'une vingtaine d'occurrences, le temps de construction et surtout le temps de maintenance dépassent presque toujours le gain, à moins que le troisième critère ne compense. Cette borne n'est pas une règle universelle, c'est un ordre de grandeur : elle sert à écarter rapidement les demandes émotionnelles, celles qui portent sur la tâche la plus détestée plutôt que sur la plus coûteuse.

STABILITÉ DE LA RÈGLE

À quelle vitesse la règle change-t-elle ? Un processus dont les conditions se modifient chaque trimestre est automatisable. Un processus dont les conditions changent au gré des négociations commerciales, sans préavis et sans traçabilité, ne l'est pas : chaque changement non répercuté produit des sorties fausses, et des sorties fausses produites en silence coûtent plus cher que la lenteur qu'on voulait supprimer.

COÛT DE L'ERREUR

Que coûte une erreur passée inaperçue ? C'est le critère qui renverse les deux autres. Un processus rare mais dont l'erreur coûte une pénalité contractuelle, une double facturation ou un client perdu justifie une automatisation que le seul volume ne justifierait jamais. Inversement, un processus fréquent dont l'erreur se rattrape en trente secondes ne mérite pas qu'on y consacre trois semaines.

LE VRAI COÛT

Ce n'est pas de construire, c'est de maintenir. Une règle qui change tous les deux mois coûte plus cher automatisée que manuelle.

TROIS PORTES, DANS CET ORDRE

Aucune des trois ne peut être sautée. Un processus non écrit ne se standardise pas, une règle instable ne se code pas, et un volume dérisoire ne rembourse pas la maintenance. La sortie « standardiser » est la plus fréquente en pratique, et la plus rentable : elle produit du gain avant même qu'un outil soit choisi.

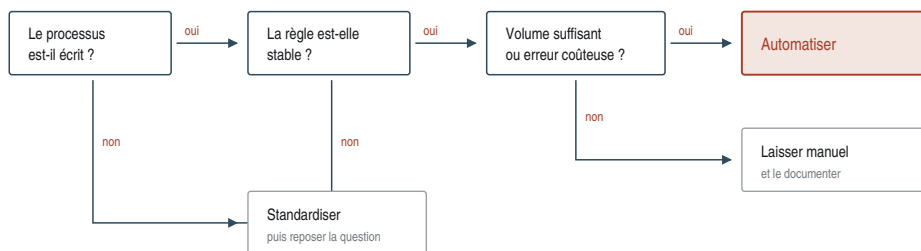


Figure 1.1 Arbre de décision préalable à tout choix d'outil. La branche qui mène à « standardiser » n'est pas un échec du projet : c'est souvent l'endroit où se trouve la plus grande part du gain, obtenue sans écrire une ligne de configuration.

1.3 Ce qu'on gagne réellement

L'automatisation produit trois gains distincts, qu'il faut compter séparément parce qu'ils ne s'adressent pas aux mêmes personnes et ne se prouvent pas de la même façon.

Les heures récupérées sont le gain le plus facile à calculer et le moins convaincant. Facile, parce qu'il suffit de multiplier un volume par une durée unitaire. Peu convaincant, parce que personne ne licencie quelqu'un après avoir automatisé une tâche : les heures libérées se redéploient, et ce redéploiement est invisible dans les comptes. Ce gain sert à ouvrir la discussion, rarement à la conclure.

Les erreurs évitées constituent le gain le plus solide et le plus difficile à chiffrer, parce qu'il faut connaître le taux d'erreur du processus manuel. Presque aucune organisation ne le mesure. Une mission sérieuse commence donc par le mesurer, sur un échantillon et pendant deux semaines, avant de toucher au moindre outil. Sans cette mesure de référence, toute affirmation de gain après coup est invérifiable, et un chiffre invérifiable ne finance pas une deuxième mission.

Le temps de cycle est le gain qui change le résultat commercial. Un devis envoyé en quatre minutes au lieu de deux jours ne fait pas économiser deux jours de salaire ; il fait gagner des affaires que la concurrence prenait. C'est le seul des trois gains qui déplace une ligne de chiffre d'affaires plutôt qu'une ligne de charges, et c'est donc celui qu'il faut mettre en avant devant un dirigeant.

1.4 Chiffrer sans mentir

Le calcul du retour sur investissement est l'endroit où la profession perd sa crédibilité. On y voit des taux de trois cents pour cent obtenus en multipliant un volume optimiste par un temps unitaire arrondi vers le haut, sans jamais

MESURE DE RÉFÉRENCE

Deux semaines d'observation avant de toucher à l'outil. Sans elles, le gain

soustraire quoi que ce soit. La formule honnête est plus courte et donne des chiffres plus petits.

annoncé après coup n'est qu'une affirmation.

$$\text{Gain net} = (V \times t \times c \times k) - (M + P)$$

V volume mensuel · t temps unitaire manuel · c coût horaire chargé · k taux de couverture

M maintenance mensuelle · P coût de plateforme mensuel

Deux termes de cette formule sont systématiquement omis, et ce sont les deux qui décident.

Le **taux de couverture** est la proportion des cas que l'automatisation traite réellement de bout en bout. Il n'atteint jamais cent pour cent. Les cas particuliers, les pièces jointes illisibles, les clients qui écrivent en trois messages successifs continuent d'arriver et continuent d'être traités à la main. Un taux de couverture de soixante-quinze pour cent est un bon résultat sur un processus réel ; annoncer cent pour cent revient à promettre une déception.

La **maintenance** existe même quand rien ne casse. Une API change de format, un identifiant expire, un partenaire modifie sa structure de réponse, une version de la plateforme retire un nœud. Le compter à zéro est la faute la plus commune. En l'absence d'historique sur l'installation concernée, on provisionne une fraction du temps de construction chaque mois, on l'inscrit dans le devis, et on la révisé avec les chiffres réels au bout de six mois. Ce que cette ligne coûte en crédibilité au moment de la vente, elle le rend au premier incident.

LA QUESTION QUI TRANCHE

Si l'automatisation tombait en panne demain et restait hors service une semaine, que se passerait-il ? Une réponse embarrassée signale un processus critique qu'il faut traiter avec le sérieux du chapitre 7. Une réponse indifférente signale un processus dont l'automatisation ne rembourse probablement pas sa propre maintenance.

1.5 Quatre situations où la réponse est non

Un consultant qui n'a jamais refusé une mission d'automatisation n'a pas de méthode, il a un carnet de commandes. Quatre situations justifient un refus argumenté, et l'argumenter correctement vaut souvent la mission suivante.

- **La règle n'est pas écrite et ne peut pas l'être.** Le processus repose sur du jugement, sur la connaissance des clients, sur des arbitrages contextuels. Ce qui est automatisable, ce sont les tâches périphériques : rassembler les informations, préparer le dossier, notifier. La décision reste humaine.

- **La règle change plus vite qu'on ne peut la recoder.** Le processus n'est pas mûr. Il faut le stabiliser d'abord, quitte à revenir dans six mois.
- **Le volume ne rembourse pas la maintenance.** Cinq occurrences par mois, quinze minutes chacune. Un tableur partagé et une convention de nommage suffisent.
- **L'erreur automatisée coûte plus cher que la lenteur.** Un traitement manuel lent mais surveillé vaut mieux qu'un traitement instantané qui envoie trois cents relances erronées avant que quiconque s'en aperçoive. Si ce cas est néanmoins retenu, il exige la totalité de l'appareil du chapitre 7 : idempotence, workflow d'erreur, alerte, et point de contrôle humain.

1.6 Standardiser d'abord

La sortie la plus fréquente de la figure 1.1 n'est ni « automatiser » ni « laisser manuel », c'est « standardiser puis repenser la question ». Elle est aussi la plus rentable, et c'est celle que personne ne vend parce qu'elle ne se facture pas en jours de développement.

Standardiser signifie ramener les variantes d'un même processus à une seule, écrite. Un formulaire unique au lieu de trois canaux de saisie. Une convention de nommage au lieu de trois. Une liste de statuts fermée au lieu d'un champ libre. Ces changements ne coûtent presque rien, produisent immédiatement une partie du gain attendu, et surtout rendent le processus automatisable ensuite. Beaucoup de projets qui échouent techniquement ont en réalité échoué à cette étape, sautée par impatience.

Le corollaire vaut d'être énoncé, parce qu'il gouverne tout le reste du livre : un processus standardisé se code en quelques heures, un processus non standardisé ne se code pas du tout, quel que soit l'outil.

Le paysage 2026 : où se situe n8n

Les comparatifs d'outils vieillissent en quelques mois. Les familles auxquelles ces outils appartiennent, elles, ne bougent pas. C'est la famille qu'il faut choisir en premier.

2.1 Quatre familles, pas quinze outils

La confusion habituelle vient de ce qu'on compare des outils qui ne résolvent pas le même problème. Quatre familles suffisent à ranger le paysage, et le choix se fait presque toujours entre familles avant de se faire entre produits.

Les plateformes d'intégration grand public. Zapier, Make, Power Automate. Hébergement fermé, catalogue d'intégrations très large, mise en route en quelques minutes, facturation à l'unité de traitement. Elles conviennent à des enchaînements courts entre applications standard, et deviennent coûteuses dès que le traitement se complexifie.

Les plateformes de workflow auto-hébergeables. n8n, Activepieces, Windmill, Node-RED. Même modèle visuel, mais le moteur tourne sur votre infrastructure, avec accès au code, aux secrets et aux journaux. Le coût se déplace de la facture vers le temps d'exploitation.

Les orchestrateurs d'exécution durable. Temporal en est le représentant le plus connu. Ils sont conçus pour des processus qui durent des jours ou des semaines, survivent au redémarrage de la machine, et reprennent exactement là où ils s'étaient arrêtés. Ce n'est pas le même problème : c'est de la fiabilité d'exécution, pas de l'intégration d'applications.

Les moteurs de pipelines et de processus. Airflow, Kestra et Dagster pour l'orchestration de traitements de données volumineux ; Camunda pour les processus métier modélisés en BPMN, avec tâches humaines, délégations et escalades. Là encore, problème différent, outillage différent.



Figure 2.1 Positionnement des familles selon deux axes seulement : la part de code que l'outil exige, et le degré de contrôle que vous gardez sur l'exécution. Les positions sont indicatives et destinées à situer des familles, non à départager des produits sur des fonctionnalités qui changent tous les trimestres.

2.2 Le modèle économique décide plus que les fonctionnalités

Sur les fonctionnalités, tous ces outils convergent. Sur la facturation, ils divergent radicalement, et c'est cette divergence qui détermine ce que vous pouvez construire sans vous ruiner.

UNITÉ FACTURÉE	OUTILS	CE QUE CELA ENCOURAGE
La tâche	Zapier	Des enchaînements courts. Chaque étape compte, donc chaque étape se négocie.
L'opération	Make	Même effet, à granularité plus fine encore.
L'exécution	n8n Cloud	Des workflows riches. Le nombre de nœuds est indifférent, seul compte le nombre de déclenchements.
Le serveur	n8n auto-hébergé, Activepieces, Windmill	Le volume devient gratuit et le temps d'exploitation devient le coût réel.

La conséquence pratique est plus importante qu'elle n'en a l'air. Sur une facturation à la tâche, on écrit des workflows courts, on renonce aux contrôles, on saute les étapes de validation parce que chacune se paie. Sur une facturation à l'exécution, ces mêmes contrôles sont gratuits, et un workflow correctement instrumenté, avec ses vérifications, sa gestion d'erreur et sa journalisation, coûte exactement le même prix qu'un workflow bâclé. Le modèle économique de la plateforme finit par sculpter la qualité de ce qu'on y construit.

Le raisonnement s'inverse en auto-hébergement. Le volume ne coûte plus rien, mais la sauvegarde, les mises à jour, les certificats, la supervision et la reprise après incident deviennent votre travail. C'est un arbitrage entre une facture et un temps, pas entre payant et gratuit.

CONSÉQUENCE DE CONCEPTION

Chez n8n, un workflow de quarante nœuds coûte le même prix qu'un workflow de trois. Ailleurs, il coûte treize fois plus.

2.3 La licence, lue correctement

n8n est distribué sous *Sustainable Use License*, que l'éditeur qualifie de *fair-code*. Ce n'est pas une licence open source au sens de l'OSI, et la confusion sur ce point produit régulièrement de mauvaises surprises en revue d'achat.

Le principe tient en une phrase : vous pouvez utiliser et modifier n8n pour vos propres besoins internes ou non commerciaux, gratuitement et sans limite de volume. Ce que la licence interdit, c'est d'en faire un produit : revendre n8n, l'offrir comme service hébergé à vos clients, ou l'intégrer dans une offre concurrente. Ces usages exigent un accord commercial négocié avec l'éditeur.

Le point qui intéresse directement le lecteur consultant est souvent mal compris dans les deux sens. La restriction sur la facturation de prestations de conseil et de support autour de n8n a été explicitement levée : construire des workflows pour un client, l'accompagner, le former et maintenir son installation ne nécessite aucun accord de licence séparé. En revanche, opérer une instance mutualisée sur laquelle vous hébergez les automatisations de plusieurs clients contre abonnement relève du service hébergé, et là il faut un accord.

Dernier détail, celui qu'on découvre en audit plutôt qu'à l'installation : les fichiers dont le chemin contient `.ee` relèvent de la licence Enterprise et non de la licence d'usage. Certaines fonctions visibles dans l'interface appartiennent à cette catégorie.

POUR UN CONSULTANT

Construire des workflows pour un client se facture librement. Héberger l'instance à sa place et la lui revendre au mois, non.

SUSTAINABLE USE LICENSE

SANS ACCORD PARTICULIER	ACCORD COMMERCIAL REQUIS
■ Auto-hébergement pour vos besoins internes ■ Volume d'exécutions illimité ■ Modification du code source ■ Prestations de conseil et d'intégration ■ Support et formation facturés	■ Revendre n8n ■ L'offrir en service hébergé à des tiers ■ L'intégrer dans un produit concurrent ■ Instance mutualisée multi-clients ■ Fonctions sous licence Enterprise

Figure 2.2 Frontière d'usage de la licence. La colonne de gauche couvre l'essentiel d'une pratique de conseil. La ligne de partage n'est pas la présence d'argent dans la transaction, mais la nature de ce qui est vendu : une prestation autour de n8n, ou n8n lui-même.

Pour situer, les alternatives directes ont des licences différentes : Activepieces est sous MIT, Node-RED sous Apache 2.0, Temporal sous MIT, Airflow et Kestra sous Apache 2.0, Windmill sous AGPLv3. Cette dernière mérite une lecture attentive : le copyleft de l'AGPL peut contraindre à ouvrir votre propre code si vous construisez un produit par-dessus. Aucune licence n'est gratuite d'obligations, elles ont simplement des obligations différentes.

2.4 L'outil sera-t-il encore là dans trois ans

C'est la question que pose le dirigeant, et elle est légitime : on ne bâtit pas ses processus critiques sur un projet qui peut disparaître.

Les faits disponibles à la date de rédaction sont plutôt rassurants. n8n a été fondée à Berlin en 2019 par Jan Oberhauser. Une levée de série C de 180 millions de dollars menée par Accel, en octobre 2025, a valorisé l'entreprise à 2,5 milliards. En mai 2026, SAP est entré au capital, portant la valorisation à 5,2 milliards, avec un accord commercial pluriannuel qui intègre n8n dans Joule Studio, l'environnement de construction d'agents de SAP. L'éditeur annonce 1,7 million de constructeurs actifs mensuels et plus de 1 400 clients grands comptes.

Une nuance mérite d'être posée, parce qu'elle distingue une lecture de communiqué d'une lecture d'analyste. L'opération SAP a pris la forme d'une cession de titres existants et non d'une émission de titres nouveaux. Le chiffre de 5,2 milliards indique donc le prix qu'un acheteur stratégique unique a accepté de payer pour une participation minoritaire, et non un apport d'argent frais au bilan. Cela ne retire rien à la solidité du signal, mais cela n'en fait pas non plus une levée de fonds.

Le vrai risque n'est de toute façon pas la disparition de l'éditeur. C'est le rythme des versions. La 2.0 a durci la plateforme et retiré des comportements sur lesquels des installations reposaient. La 3.0, attendue en octobre 2026, retire des nœuds entiers et impose Docker en auto-hébergement. Un parc d'automatisations non maintenu se dégrade en deux versions majeures. C'est un risque opérationnel, il se gère, et c'est l'objet du chapitre 23.

2.5 Quand n8n est le mauvais choix

Cinq situations où un autre outil vaut mieux, et où le dire vaut mieux que de forcer.

- **Processus longs et repris.** Un traitement qui s'étend sur des jours ou des semaines, avec reprise exacte après panne et garanties de progression, relève d'un orchestrateur durable comme Temporal.
- **Volumes de données importants.** Des millions de lignes à transformer relèvent d'Airflow, de Kestra ou d'une base de données, pas d'un moteur qui matérialise chaque enregistrement en item.
- **Processus humains complexes.** Validations à plusieurs niveaux, délégations, escalades sur délai, conformité à un modèle BPMN : c'est le terrain de Camunda.
- **Traitement enregistrement par enregistrement à très fort débit.** Le modèle d'itération de n8n, décrit au chapitre suivant, se retourne contre vous. Une requête SQL fait le travail en une fraction du temps.

NUANCE

L'opération SAP est une cession secondaire, pas une levée. Elle dit ce qu'un acheteur a payé, pas ce que l'entreprise a encaissé.

- **Produit revendu.** Si l'automatisation doit être intégrée dans une offre commercialisée, la licence impose un accord ou un autre outil.

Reste le domaine où n8n est difficile à battre, et il est vaste : les processus d'intégration entre applications, de durée courte, à logique riche, mêlant appels d'API, transformations, décisions et désormais agents, sur une infrastructure dont on garde le contrôle. C'est ce domaine que traite le reste de ce livre.

Anatomie de n8n

Presque toutes les erreurs durables commises dans n8n viennent d'une mauvaise compréhension d'une seule chose : ce qui circule entre deux nœuds. Ce chapitre traite cette chose en premier, et tout le reste en découle.

3.1 Le vocabulaire minimal

Cinq mots suffisent, et il vaut la peine de les fixer parce que la documentation officielle et la communauté ne les emploient pas toujours de la même façon.

Un **workflow** est un graphe de nœuds reliés par des connexions. Un **nœud** est une unité de traitement : il reçoit des données, fait quelque chose, produit des données. Une **exécution** est un parcours complet de ce graphe, depuis son déclenchement jusqu'à son terme, et c'est l'unité que la plateforme facture. Un **déclencheur** est le nœud par lequel une exécution commence ; un workflow sans déclencheur ne peut pas tourner en production. Enfin, un workflow existe en deux états distincts depuis la version 2.0 : la version enregistrée, sur laquelle vous travaillez, et la version publiée, celle qui tourne réellement. Enregistrer ne met plus en production.

PIÈGE DE LA VERSION 2.0

Un correctif enregistré mais non publié ne change rien au comportement de l'installation. C'est la cause la plus fréquente du « j'ai pourtant corrigé le bug » chez les équipes qui viennent de la version 1.

3.2 Le modèle de données

Tout ce qui circule sur une connexion entre deux nœuds a la même forme, sans exception : un tableau d'objets. Chaque objet de ce tableau s'appelle un *item*. Chaque item porte obligatoirement une clé `json`, qui contient les données structurées, et facultativement une clé `binary`, qui contient les fichiers.

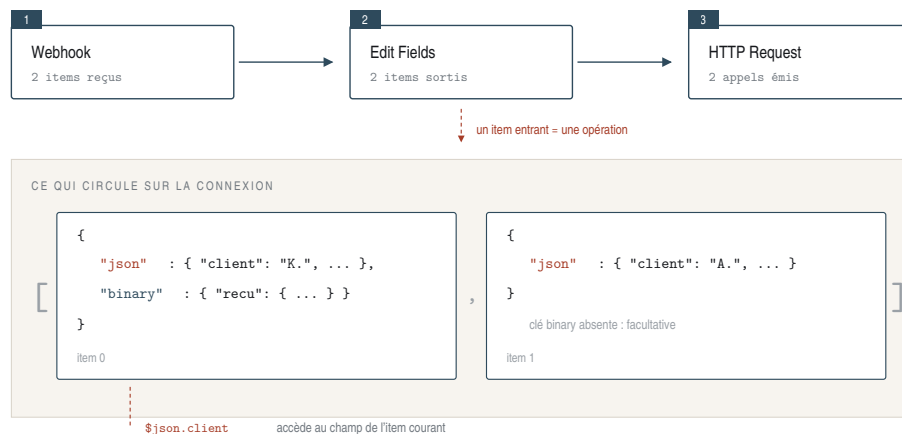


Figure 3.1 Structure d'un item et propagation le long d'une chaîne de nœuds. Tout ce qui circule entre deux nœuds est un tableau d'objets, chacun portant une clé `json` obligatoire et une clé `binary` facultative. Un nœud recevant deux items exécute son opération deux fois : c'est ce comportement d'itération implicite, et non une boucle explicite, qui gouverne le volume d'appels sortants.

Cette uniformité est la principale qualité du modèle. Elle signifie qu'un nœud n'a jamais besoin de savoir d'où viennent ses données : un webhook, une base de données, un fichier lu sur disque ou un agent produisent tous la même structure. Elle explique aussi la principale difficulté du modèle, qui fait l'objet de la section suivante.

Un point de vocabulaire mérite d'être posé tout de suite parce qu'il est source de confusion permanente. Le mot « donnée » recouvre en réalité deux niveaux : le tableau, qui détermine combien de fois le nœud s'exécutera, et le contenu de chaque item, qui détermine ce qu'il fera. Confondre les deux produit des workflows qui fonctionnent parfaitement sur un item de test et se comportent de façon inattendue dès qu'il y en a douze.

À RETENIR

La question à se poser devant tout nœud n'est pas « que fait-il », mais « combien d'items entrent, combien sortent ».

3.3 L'itération implicite

Un nœud qui reçoit trois items exécute son opération trois fois. Il n'y a pas de boucle à écrire, pas d'option à cocher : le comportement est implicite, et c'est la règle la plus lourde de conséquences de toute la plateforme.

Un nœud HTTP Request qui reçoit cent items émet cent requêtes. Un nœud d'envoi de message qui reçoit cent items envoie cent messages. Ce comportement est exactement ce qu'on veut la plupart du temps, et exactement ce qu'on ne veut pas quand un filtre mal réglé en amont laisse passer la table entière au lieu des trois lignes attendues. Les incidents les plus mémorables du métier tiennent tous dans cette phrase.

Trois conséquences pratiques en découlent, qui reviendront tout au long du livre. Le nombre de requêtes sortantes se lit sur le canevas avant toute exécution, en comptant les items à chaque étape. Les limitations de débit des API se dimensionnent sur ce nombre, pas sur le nombre de déclenchements. Et tout

nœud qui envoie quelque chose vers l'extérieur mérite qu'on regarde le compte d'items juste avant lui, systématiquement, avant la première mise en production.

3.4 Les familles de nœuds

Le catalogue compte plus d'un millier d'entrées, mais il n'y a que quatre familles, et connaître la famille d'un nœud renseigne mieux sur son comportement que sa documentation.

Les déclencheurs ouvrent une exécution. Ils réagissent à un événement extérieur, à un appel entrant, à une horloge, ou à l'appel d'un autre workflow. Un workflow n'en a qu'un actif à la fois en pratique.

Les nœuds d'application parlent à un service précis. Ce sont les plus nombreux et les moins intéressants conceptuellement : quand l'un d'eux ne couvre pas l'opération voulue, le nœud HTTP Request générique prend le relais en réutilisant les mêmes identifiants enregistrés.

Les nœuds de cœur ne parlent à personne. Ils portent la logique : conditions, aiguillage, filtrage, tri, fusion, découpage, temporisation, transformation, réponse, arrêt en erreur. Ce sont eux qui font la différence entre un enchaînement et un programme, et le chapitre 5 leur est consacré.

Les nœuds en grappe forment la famille la plus récente et la plus déroutante. Au lieu d'un nœud isolé, on assemble un nœud racine et un ou plusieurs sous-nœuds qui étendent ses capacités : un modèle de langage, une mémoire, des outils, un magasin vectoriel. C'est la structure de toute la partie IV. Elle apporte avec elle une asymétrie qu'il faut connaître dès maintenant, car elle produit des bugs silencieux : dans un nœud ordinaire, une expression est ré-évaluée pour chaque item ; dans un sous-nœud, elle résout systématiquement vers le premier item, quel que soit le nombre reçu.

PIÈGE N° 1

Dans un sous-nœud, une expression résout toujours vers le premier item. Dans un nœud normal, elle résout item par item.

3.5 Les expressions

Une expression est du JavaScript évalué à l'exécution, encadré par deux paires d'accolades. Elle sert à remplir un paramètre à partir des données qui circulent plutôt qu'avec une valeur figée.

```
{{ $json.client }}
{{ $json.montant * 1.18 }}
{{ $('Webhook').item.json.reference }}
{{ $now.toISO() }}
```

Quelques variables couvrent la quasi-totalité des besoins. `$json` désigne le contenu de l'item courant ; `$binary` ses fichiers ; `$('Nom du nœud')` permet de remonter aux données d'un nœud antérieur, ce qui est indispensable dès qu'une

branche a transformé la structure ; `$now` donne l'instant courant ; `$execution` et `$workflow` exposent les identifiants utiles pour la journalisation.

Deux recommandations valent d'être prises tôt. La première est de préférer `$('Nom du nœud')` aux références positionnelles : un renommage casse moins de choses qu'un déplacement de nœud. La seconde est de ne pas mettre de logique dans les expressions. Une expression qui dépasse une ligne appartient à un nœud Code ou à un nœud de cœur, où elle sera visible, testable et compréhensible par la personne qui reprendra le workflow. Une condition ternaire imbriquée cachée dans un champ de paramètre est invisible à la relecture du canevas.

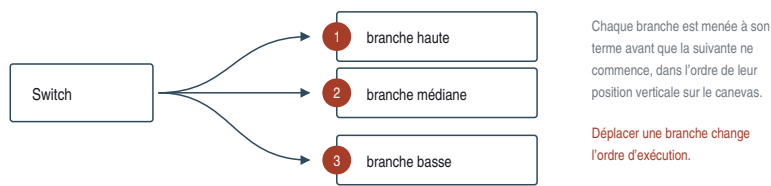
Compatibilité. L'assistant `$getPairedItem` est retiré dans la version 3.0. Les workflows qui l'utilisent doivent passer par les mécanismes de chaînage décrits au chapitre 5. Le chapitre 23 donne la procédure de repérage sur un parc existant.

3.6 L'ordre d'exécution

Quand un nœud ouvre plusieurs branches, l'ordre dans lequel elles s'exécutent n'est ni aléatoire ni parallèle. Pour tout workflow créé depuis la version 1.0, n8n termine entièrement une branche avant d'entamer la suivante, et il classe les branches selon leur position sur le canevas, de la plus haute à la plus basse. À hauteur égale, la plus à gauche passe en premier.

Cette règle a l'air anodine et ne l'est pas. Elle signifie que la disposition visuelle de vos nœuds a une valeur sémantique : déplacer une branche de deux centimètres vers le haut change l'ordre d'exécution du workflow. Dès que deux branches écrivent dans le même enregistrement, ou que l'une dépend d'un effet produit par l'autre, cette position devient une décision de conception et non un choix esthétique. Le comportement est modifiable dans les réglages du workflow, mais il vaut mieux composer avec plutôt que le contourner.

A - ORDRE DES BRANCHES



B - ÉTATS D'UNE EXÉCUTION

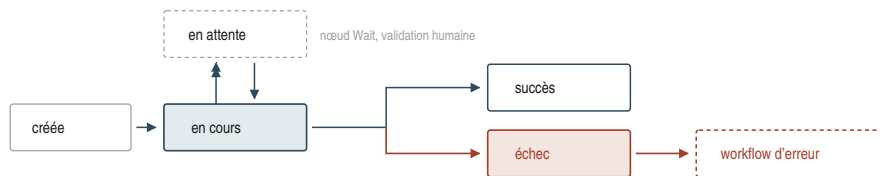


Figure 3.2 Deux règles d'exécution qu'il faut connaître avant d'écrire quoi que ce soit. En haut, l'ordre des branches, gouverné par la position sur le canevas. En bas, les états par lesquels passe une exécution, et le point d'accroche du workflow d'erreur, qui fait l'objet du chapitre 7.

3.7 Ce que n8n ne retient pas pour vous

Par défaut, une exécution ne sait rien des exécutions précédentes. C'est une propriété saine, et c'est aussi la source d'une déception récurrente : le compteur qu'on croyait incrémenter, le marqueur qu'on croyait avoir posé, la liste qu'on croyait avoir enrichie n'existent pas d'une exécution à l'autre.

Trois mécanismes permettent de persister quelque chose, avec des usages distincts. Les **data tables** stockent des données structurées à l'intérieur de l'instance, à raison de 200 Mio par défaut pour l'ensemble d'une installation, avec avertissement à 80 % du plafond et erreurs d'exécution au-delà. Elles sont faites pour les tables de correspondance, les marqueurs anti-doublon et les jeux de test, et elles ne sont pas accessibles depuis un nœud Code. Les **variables** conviennent aux valeurs courtes de configuration, partagées entre projets. Toute autre persistance appelle une base de données externe, et le chapitre 7 explique pourquoi le choix de la clé importe davantage que le choix du stockage.

3.8 Découper : les sous-workflows

Un workflow de soixante nœuds n'est pas impressionnant, il est ingérable. Le découpage en sous-workflows appelés par un workflow parent est le seul mécanisme de modularité de la plateforme, et il faut l'adopter tôt plutôt que d'y venir en catastrophe.

Il sert trois fins. Réutiliser une logique commune à plusieurs processus, typiquement la normalisation d'un client ou l'écriture dans l'outil de gestion. Isoler une partie fragile, pour qu'un échec sur elle ne fasse pas tomber tout le traitement. Et rendre lisible : un canevas parent de dix nœuds dont trois ap-

pellent des sous-workflows nommés se comprend en une minute, là où le même traitement à plat demande un quart d'heure.

Le prix à payer est le passage de contexte. Le sous-workflow ne voit que ce qu'on lui transmet, et cette contrainte, agaçante au début, est en réalité la même que celle d'une fonction : elle force à définir une interface. Le chapitre 4 en fait un principe de conception plutôt qu'une contrainte subie.

CE QUE LE CHAPITRE SUIVANT AJOUTE

La partie II passe de la description au métier. Le chapitre 4 traduit un processus écrit en diagramme de workflow, avec des règles de découpage explicites ; le chapitre 5 reprend le modèle d'items pour traiter le chaînage, la fusion et l'agrégation ; le chapitre 6 couvre les déclencheurs et les webhooks ; le chapitre 7 traite l'échec, qui est le seul sujet dont dépend la mise en production.

Du processus métier au diagramme de workflow

PISTE PRATICIEN ET PISTE DÉCIDEUR

Entre la page qui décrit le processus et le canevas qui l'exécute, il y a une traduction. Ce chapitre en donne les règles, et elles se posent sur papier, avant d'ouvrir n8n.

4.1 Une notation de travail

Le diagramme dont il est question ici n'est pas un diagramme n8n. C'est un schéma de travail, dessiné avec le client, qui doit tenir sur une feuille et se comprendre sans formation. Cinq symboles suffisent, et il vaut mieux s'y tenir que d'emprunter une notation formelle que personne ne saura relire.

Un **déclencheur** ouvre le processus. Une **transformation** change la forme des données sans toucher au monde extérieur. Une **décision** ouvre plusieurs chemins. Un **effet externe** écrit quelque part, envoie quelque chose, change l'état d'un système tiers. Un **point de reprise** marque un endroit où le traitement peut redémarrer sans dommage.

La distinction qui compte réellement est celle entre transformation et effet externe. Une transformation ratée se recommence sans conséquence. Un effet externe raté laisse une trace : un message parti, une ligne créée, un paiement enregistré. Sur le schéma, les effets externes reçoivent un double trait, et cette convention typographique fait plus pour la qualité d'un projet que n'importe quel outil, parce qu'elle rend visible d'un coup d'œil tout ce qui devra être rendu rejouable au chapitre 7.

4.2 Quatre règles de découpage

Le passage du schéma au canevas se joue sur le découpage. Quatre règles suffisent, et elles évitent la quasi-totalité des workflows illisibles.

Un **workflow**, un **déclencheur**, un **résultat métier**. Si vous ne pouvez pas nommer le workflow par ce qu'il produit, il en contient deux. « Traiter une commande entrante » est un bon nom. « Gestion clients » n'en est pas un.

POURQUOI PAS BPMN

Trop lourd pour ce qu'on fait, et illisible par le client. Cinq symboles suffisent.

Tout effet externe utilisé deux fois devient un sous-workflow. L'écriture dans l'outil de gestion, la notification au client, la normalisation d'un numéro de téléphone : dès qu'un traitement apparaît dans deux processus, il sort et devient callable. La duplication d'un effet externe est la garantie de le corriger à un seul endroit sur deux le jour où l'API change.

La frontière d'un sous-workflow est son contrat. Il ne voit que ce qu'on lui passe. Cette contrainte, ressentie comme une gêne au début, force à définir une interface stable, et c'est elle qui permet de remplacer l'implémentation sans toucher aux appelants.

Le chemin nominal se lit horizontalement, les exceptions descendent. Cette règle est purement visuelle et elle a une conséquence technique directe, vue au chapitre 3 : l'ordre d'exécution des branches suit leur position verticale. Placer le chemin nominal en haut, c'est aussi le faire passer en premier.

TEST DE DÉCOUPAGE

Un sous-workflow dont on ne sait pas énoncer le contrat d'entrée en une phrase n'est pas un sous-workflow, c'est une coupure arbitraire.

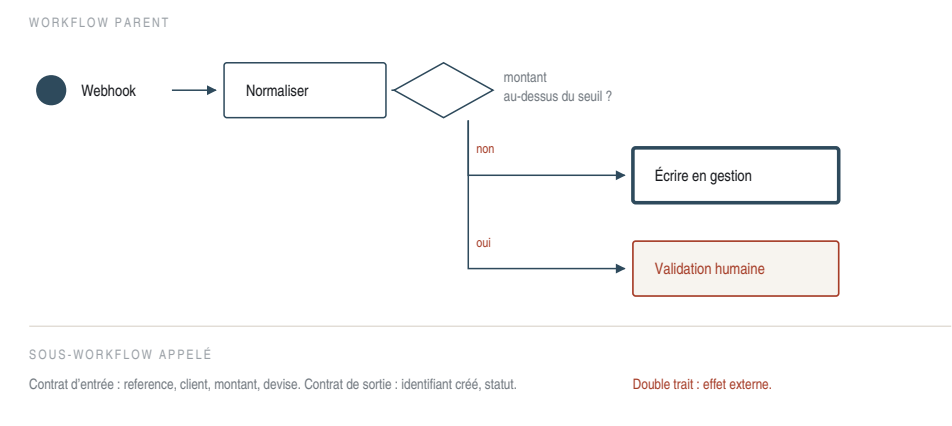


Figure 4.1 Un processus décomposé selon les quatre règles. Le chemin nominal occupe la ligne haute et passe donc en premier à l'exécution. Le seul effet externe est signalé par un double trait, ce qui identifie immédiatement le point qui devra être rendu rejouable.

4.3 Nommer

Les noms de nœuds par défaut sont une dette technique contractée en trois secondes. Un canevas où figurent « HTTP Request », « HTTP Request1 » et « HTTP Request2 » est illisible, et il le devient davantage encore quand une expression y fait référence.

La convention retenue dans ce livre tient en une règle : un nœud se nomme par ce qu'il fait, à l'impératif, dans le vocabulaire du métier et non dans celui de l'outil. « Récupérer le client », « Créer la facture », « Notifier le commercial ». La conséquence pratique est immédiate à la lecture d'une expression : `$('Récupérer le client')` se comprend sans ouvrir le nœud, là où `$('HTTP Request2')` oblige à aller voir.

4.4 Concevoir idempotent au tableau blanc

L'idempotence se décide au moment du schéma, pas au moment du débogage. La question à poser est unique et elle porte sur chaque effet externe : si ce traitement est rejoué deux fois avec les mêmes données d'entrée, que se passe-t-il ?

Répondre exige d'identifier une **clé naturelle**, c'est-à-dire une valeur qui identifie de façon stable l'événement traité et qui vient de l'émetteur. La référence de transaction d'un prestataire de paiement, l'identifiant de message d'un webhook, le numéro de commande. Cette clé ne doit jamais être fabriquée par vous : un horodatage de réception ou un identifiant tiré au hasard change à chaque rejeu et ne protège de rien.

LA QUESTION À POSER AU CLIENT, EN UNE PHRASE

Quelle valeur, dans ce que vous recevez, identifie de manière unique et définitive l'événement ? Si personne ne sait répondre, l'idempotence n'est pas implémentable, et il faut le dire avant de commencer plutôt qu'après le premier doublon en production.

4.5 Où placer les points de contrôle humains

Un point de contrôle humain coûte du temps de cycle et en rend une partie du bénéfice. Il se place donc là où il rapporte, c'est-à-dire immédiatement avant les effets externes irréversibles ou coûteux : un envoi vers un client, un document engageant, un mouvement d'argent, une suppression.

Trois principes gouvernent leur usage. Le contrôle doit arriver avec tout le contexte nécessaire pour décider en dix secondes, sinon il devient un goulot d'étranglement que les gens contournent. Il doit avoir une échéance et un comportement par défaut à l'expiration, sinon les exécutions s'accumulent en attente. Et il doit être placé sur une fraction des cas, pas sur tous : un seuil de montant ou un score de confiance réduit souvent le volume de contrôle de quatre-vingts pour cent sans réduire la protection.

4.6 La revue de conception en quinze minutes

Avant d'ouvrir l'éditeur, le schéma passe six questions. Elles paraissent triviales et elles économisent des semaines.

- Qu'est-ce qui déclenche, et combien de fois par jour au maximum ?
- Quels sont les effets externes, et lequel est le plus coûteux à rejouer ?
- Quelle est la clé naturelle de l'événement ?
- Que se passe-t-il si l'appel du milieu échoue et que celui d'avant a déjà écrit ?
- Qui reçoit l'alerte en cas d'échec, et sur quel canal le verra-t-il vraiment ?

- Quels cas ne seront pas couverts, et qui les traitera à la main ?

La dernière question est celle que l'on oublie et c'est celle qui détermine le taux de couverture du chapitre 1. Un processus dont on a nommé les exceptions dès la conception se livre avec une procédure manuelle assumée. Un processus dont on ne les a pas nommées se livre avec une surprise.

Données, expressions et chaînage d'items

Le chapitre 3 a posé la forme de ce qui circule. Celui-ci traite de ce qu'on en fait, et surtout de la seule chose que la plateforme ne rend pas visible : la trace qui relie un item de sortie à l'item d'entrée dont il provient.

5.1 Six verbes suffisent

Le catalogue des nœuds de transformation est long, mais six opérations couvrent l'essentiel, et ce qui les distingue n'est pas ce qu'elles font au contenu : c'est ce qu'elles font au *nombre* d'items. Raisonner en cardinalité plutôt qu'en fonctionnalité est le raccourci le plus rentable de tout l'apprentissage de n8n.

VERBE	ENTRÉE	SORTIE	CARDINALITÉ
Edit Fields	● ● ●	→ ● ● ●	n vers n, contenu modifié
Filter	● ● ●	→ ●	n vers m, m inférieur ou égal à n
Split Out	●	→ ● ● ● ●	1 vers n, une liste éclatée
Aggregate	● ● ●	→ ●	n vers 1, un tableau regroupé
Merge	● ● ● ● ●	→ ● ● ● ● ●	n plus m vers k
Loop Over Items	● ● ● ●	→ ● ●	n vers lots successifs

Figure 5.1 Les six verbes classés par ce qu'ils font au nombre d'items. Lire un canevas consiste d'abord à suivre cette colonne : c'est elle qui prédit le nombre d'appels sortants, donc le coût et le risque.

Edit Fields conserve la cardinalité et change le contenu. C'est le nœud le plus utilisé et le plus sous-estimé : il sert autant à construire une structure qu'à la documenter, en donnant des noms explicites aux champs juste avant un appel externe.

Filter réduit. C'est le nœud à surveiller en priorité avant tout effet externe, parce qu'un filtre mal réglé ne produit pas d'erreur : il produit trop d'items, et la conséquence se lit sur la facture ou dans la boîte de réception des clients.

Split Out éclate un tableau contenu dans un champ en autant d'items. **Aggregate** fait l'inverse et regroupe. Ces deux-là fonctionnent en paire et sont la réponse à la question la plus fréquente des débutants : comment traiter une liste imbriquée.

Loop Over Items découpe le flux en lots successifs. Il ne sert presque jamais à « faire une boucle », puisque l'itération est déjà implicite ; il sert à cadencer, ce qui est le sujet du chapitre 6.

5.2 Merge, en détail

Le nœud Merge mérite un traitement séparé parce qu'il est le seul dont le mauvais réglage produit des résultats plausibles mais faux. Quatre modes existent.

Append empile : les items de l'entrée 1, puis ceux de l'entrée 2, et ainsi de suite. Le jeu de données devient plus long. C'est l'union, et c'est aussi la source la plus commune de doublons.

Combine joint : le jeu de données devient plus large. La variante par champs correspondants reproduit les jointures SQL, et le réglage du type de sortie décide de tout. *Keep Matches* est une jointure interne, *Keep Everything* une jointure externe, *Enrich Input 1* une jointure à gauche qui préserve tous les enregistrements de la première entrée. Deux options gouvernent les collisions : la priorité en cas de valeurs concurrentes, et le choix entre fusion profonde, qui descend dans les objets imbriqués, et fusion de surface, qui s'arrête au premier niveau.

SQL Query permet d'écrire la jointure à la main, les entrées étant exposées sous les noms `input1`, `input2` et suivants. **Choose Branch** ne fusionne rien : il attend que les branches se terminent et laisse passer celle qu'on désigne, ce qui en fait un outil de synchronisation plutôt que de fusion.

5.3 Le chaînage d'items

Voici le sujet que la plateforme ne montre pas et dont dépendent la moitié des messages d'erreur incompréhensibles.

Quand un nœud produit un item, n8n conserve un lien vers l'item d'entrée dont il provient. C'est ce lien qui permet à une expression comme `$('Webhook').item.json.reference` de savoir *quel* item du webhook correspond à l'item courant, dix nœuds plus loin. Sans lui, la question n'aurait pas de réponse dès que le flux a été filtré, réordonné ou fusionné.

RAPPEL

Merge attend l'exécution de toutes ses entrées connectées avant de produire quoi que ce soit.

LE MESSAGE

La chaîne se rompt à trois endroits, et connaître ces trois endroits fait gagner des heures de débogage.

- **Après un Merge, du côté des entrées secondaires.** Les items venus de la première entrée conservent leur lien, ceux des entrées suivantes le perdent fréquemment. C'est le cas le plus retors, parce qu'il fonctionne à moitié : l'expression est correcte pour les premiers items et échoue pour les autres.
- **Après une agrégation.** Un item de sortie provient de plusieurs items d'entrée : il n'y a plus d'origine unique à désigner.
- **Après un nœud Code qui construit ses sorties de zéro.** Le lien n'existe que si on le rétablit explicitement.

« Can't determine which item to use »
signale une chaîne rompue, pas une
expression fausse.

Le remède est le même dans les trois cas et il est structurel plutôt que technique : ne comptez pas sur la remontée à distance pour les valeurs dont vous aurez besoin plus loin. Transportez-les. Un nœud Edit Fields placé juste après le déclencheur, qui recopie la référence, l'identifiant client et l'horodatage dans des champs stables, rend inutiles les trois quarts des expressions de remontée et survit à toutes les fusions.

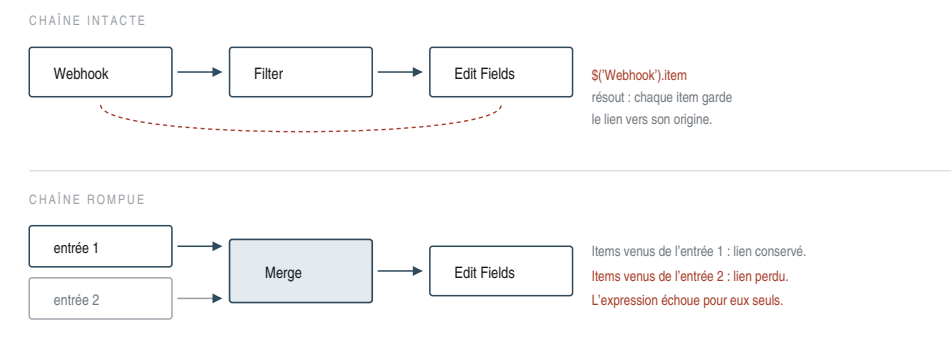


Figure 5.2 Le chaînage d'items et son point de rupture le plus fréquent. La panne est partielle, donc elle passe les tests : un jeu d'essai qui n'alimente que la première entrée du Merge ne révèle rien.

5.4 Quand écrire du code

Le nœud Code est utile et il est surutilisé. La ligne de partage est simple à énoncer : le code sert aux transformations que les nœuds ne savent pas faire, pas aux transformations qu'on ne sait pas encore faire avec eux.

Trois cas le justifient. Un calcul non trivial sur le contenu d'un item. Une restructuration profonde qu'aucune combinaison de Split Out et d'Aggregate ne produit lisiblement. Une normalisation réutilisée partout, qui gagne à vivre dans un sous-workflow appelé plutôt que d'être recopiée.

Trois cas ne le justifient pas, et ce sont ceux qu'on rencontre. Faire un appel HTTP, parce que le nœud dédié gère les identifiants, les tentatives et le cadencement. Faire une boucle, parce que l'itération est implicite. Cacher une logique métier, parce qu'elle devient invisible sur le canevas et que la personne qui reprendra le workflow ne pensera pas à ouvrir le nœud.

Depuis la version 2.0. Le nœud Code s'exécute dans un environnement isolé et n'a plus accès aux variables d'environnement de l'instance. Les secrets passent désormais par les identifiants enregistrés ou par un coffre externe, ce que traite le chapitre 9. Les tables de données ne sont pas non plus accessibles depuis le code.

5.5 Les pièges de cardinalité

Quatre situations reviennent assez souvent pour mériter d'être nommées.

Le nœud qui ne reçoit rien. Un nœud sans item d'entrée ne s'exécute pas, et la branche s'arrête en silence. Ce n'est pas une erreur, c'est un flux vide, et c'est la première chose à vérifier devant un workflow qui « ne fait rien ».

Le nœud qui reçoit tout. Un filtre inversé ou une condition trop large, et l'opération part sur le jeu complet. Le compteur d'items affiché sur chaque connexion est la seule protection réelle.

Le sous-nœud qui ne voit que le premier item. Rappelé au chapitre 3, il produit un résultat cohérent et faux : tous les items reçoivent le traitement calculé pour le premier.

L'agrégation prématurée. Regrouper avant un appel externe transforme cent appels en un seul, ce qui est parfois exactement le but et parfois la perte de la granularité dont on aura besoin pour rejouer un cas précis. Le chapitre 7 explique pourquoi cette granularité vaut souvent son coût.

Déclencheurs, webhooks et intégrations

Le déclencheur est le seul nœud dont l'échec ne produit aucune trace : quand il ne se déclenche pas, il ne se passe rien, et rien ne signale que rien ne s'est passé.

6.1 Recevoir ou aller chercher

Deux façons d'apprendre qu'un événement s'est produit. Le service vous prévient, ou vous lui demandez régulièrement.

Le **webhook** est la voie à privilégier quand elle existe. L'événement arrive dans la seconde, une exécution correspond à un événement réel, et la facture suit l'activité plutôt que l'horloge. En contrepartie, il faut une adresse publique joignable, et surtout il faut accepter que l'émetteur ne garantisse pas toujours la livraison : certains services ne réessaient pas, d'autres réessaient plusieurs fois, ce qui pose exactement le problème d'idempotence du chapitre suivant.

Le **polling** convient quand le service n'émet rien, ou quand la fraîcheur ne compte pas. Son défaut n'est pas la latence, c'est le volume : l'immense majorité des exécutions ne trouve rien à traiter et se paie quand même. Un intervalle honnêtement choisi, aligné sur le besoin métier plutôt que sur la peur de rater quelque chose, divise souvent la facture par cinq.

6.2 Le nœud Webhook

Quatre réglages décident du comportement, et le troisième est celui que l'on découvre trop tard.

Le **chemin** et la **méthode** définissent l'adresse. Chaque nœud webhook expose deux adresses distinctes, une de test et une de production, et confondre les deux est le premier rite de passage de tout débutant : l'adresse de test ne fonctionne que pendant qu'on écoute depuis l'éditeur.

COÛT CACHÉ DU POLLING

Une interrogation toutes les cinq minutes fait près de neuf mille exécutions par mois, pour une poignée d'événements réels.

Le **mode de réponse** détermine ce que reçoit l'appelant et quand. Répondre immédiatement libère l'émetteur avant que le traitement commence, ce qui est presque toujours le bon choix quand le service qui appelle impose un délai court. Répondre à la fin du dernier nœud renvoie le résultat du traitement, au prix de faire attendre l'émetteur. Répondre via un nœud dédié donne le contrôle complet du corps et du code de statut.

L'**authentification** propose une authentification basique, une authentification par en-tête et une validation de jeton JWT. Ces trois options protègent une adresse que vous contrôlez des deux côtés. Elles ne valident pas les signatures des grands émetteurs, et confondre les deux sujets est l'erreur du paragraphe suivant.

6.3 Vérifier l'authenticité

Une adresse de webhook est publique. Quiconque la connaît peut lui envoyer ce qu'il veut, et votre workflow le traitera comme un événement légitime. La question n'est donc pas de cacher l'adresse mais de vérifier la provenance.

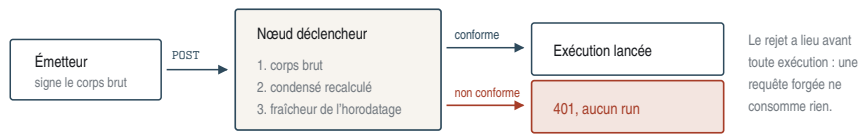
La bonne nouvelle est qu'un nombre croissant de nœuds déclencheurs dédiés le font désormais nativement. Le nœud déclencheur Stripe, depuis les versions 2.25.7 et 2.26.2, vérifie l'en-tête de signature dès qu'un secret est renseigné sur l'identifiant, et rejette avec un code 401 toute requête non signée, forgée ou vieille de plus de cinq minutes, sans exécuter le workflow. Le déclencheur Slack accepte de la même façon un secret de signature. La règle pratique en découle : quand un nœud déclencheur dédié existe pour le service, il vaut mieux que le nœud webhook générique, et son secret doit être renseigné.

Quand il faut le faire à la main, trois détails causent l'essentiel des échecs. Le calcul porte sur le **corps brut** de la requête, pas sur le JSON reparsé, dont l'ordre des clés et les espaces diffèrent. Le **format du condensé** varie selon les émetteurs, certains attendant de l'hexadécimal et d'autres du base64. Et le **contenu signé** n'est pas toujours le corps seul : Stripe signe la concaténation de l'horodatage, d'un point, et du corps brut. Une implémentation qui ignore l'un de ces trois points produit une signature qui ne correspond jamais, sans indiquer pourquoi.

ERREUR CLASSIQUE

Choisir l'option JWT pour vérifier une signature Stripe ou GitHub. Ce n'est pas un vérificateur HMAC, et cela échouera toujours.

REQUÊTE ENTRANTE



LES TROIS CAUSES D'ÉCHEC EN VÉRIFICATION MANUELLE

Corps reparsé au lieu du corps brut : condensé en hexadécimal quand l'émetteur attend du base64 : contenu signé incomplet, l'horodatage étant parfois concaténé au corps avant le calcul.

Figure 6.1 Anatomie d'une requête webhook vérifiée. Le point important est la position du rejet : avant le démarrage de l'exécution, donc sans consommer de quota et sans laisser de trace exploitable par un attaquant.

6.4 Déclencheurs planifiés

Un déclencheur planifié paraît simple et pose deux questions qu'on néglige.

La première est celle du **fuseau**. Une planification quotidienne s'exécute selon le fuseau de l'instance, qui n'est pas nécessairement celui du client. Une écriture comptable datée du mauvais jour se découvre à la clôture du mois, et c'est tard.

La seconde est celle de la **fenêtre**. Un traitement planifié qui interroge « ce qui a changé depuis la dernière fois » doit savoir ce que signifie la dernière fois. Se fier à l'heure d'exécution précédente échoue au premier redémarrage manqué : la fenêtre saute et les enregistrements de l'intervalle sont perdus pour toujours. La solution robuste consiste à faire porter la fenêtre par une valeur persistée, un horodatage maximal traité, plutôt que par le calendrier.

6.5 Le nœud HTTP Request

Quand aucun nœud dédié ne convient, le nœud HTTP Request générique fait tout, et il sait davantage de choses qu'on ne lui en demande d'ordinaire.

Il réutilise les **identifiants enregistrés** des services connus : en choisissant un type d'authentification prédéfini, on récupère l'identifiant déjà créé pour le nœud dédié, sans recopier de clé dans un en-tête. C'est le moyen le plus simple d'éviter des secrets en clair dans un paramètre.

Il gère la **pagination** nativement, ce qui évite d'écrire une boucle. Il gère le **cadencement** par une option de traitement par lots, avec un nombre d'éléments par lot et un intervalle entre les lots, qui reproduit à l'intérieur d'un seul nœud ce que l'on faisait auparavant avec un Loop Over Items et un Wait. Et il accepte, comme tous les nœuds, un réglage de **nouvelle tentative en cas d'échec**, avec un nombre de tentatives et une attente entre elles.

6.6 Cadencer

L'itération implicite du chapitre 3 rencontre ici les limites de débit des API. Trois leviers, à choisir dans cet ordre.

- **Réduire le nombre d'items** avant l'appel. C'est toujours la meilleure réponse : filtrer, dédoublonner, agréger. Un appel évité ne coûte rien et ne peut pas échouer.
- **Espacer** par l'option de lots du nœud HTTP Request, en réglant l'intervalle sur la limite documentée du fournisseur. Un appel par seconde autorisé se traduit par un lot de un et un intervalle de mille millisecondes.
- **Réessayer** avec une attente supérieure à la fenêtre de limitation, pour absorber les rejets qui passent malgré tout.

L'ordre importe. Beaucoup d'équipes commencent par la nouvelle tentative, qui est le levier le plus faible : elle transforme un problème de conception en un problème de patience, et ne réduit ni la facture ni la durée.

Erreurs, reprises et idempotence

PISTE PRATICIEN ET PISTE DÉCIDEUR

C'est le chapitre qui sépare une démonstration d'un système qu'on ose facturer. Tout ce qui précède décrit ce qui se passe quand tout va bien ; celui-ci décrit le reste, qui arrive plus souvent qu'on ne le prévoit.

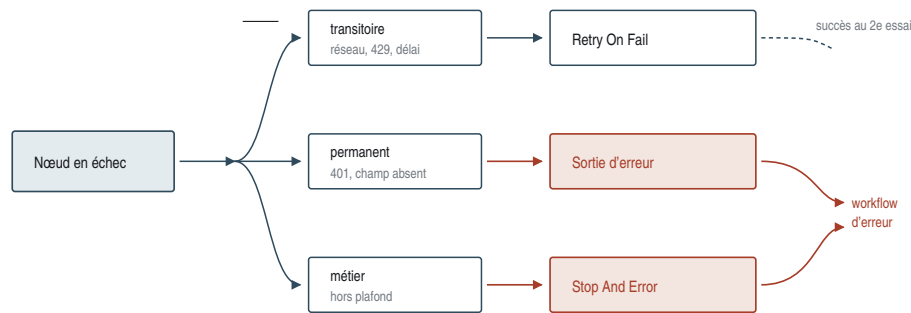
7.1 Trois familles d'échec

Traiter toutes les erreurs de la même façon revient à ne traiter aucune correctement. Trois familles, trois réponses.

L'échec transitoire disparaît tout seul : coupure réseau, service momentanément indisponible, limite de débit atteinte, délai dépassé. Il se traite par une nouvelle tentative, et l'écrasante majorité des incidents relève de cette famille. C'est aussi la famille qui, mal traitée, produit les doublons.

L'échec permanent ne disparaîtra pas : identifiant expiré, champ obligatoire absent, ressource supprimée, format refusé. Réessayer ne sert à rien et retarde l'alerte. Il se traite par une sortie d'erreur et une notification.

L'échec métier n'est pas une erreur technique : le montant dépasse le plafond autorisé, le client est inconnu, la pièce jointe est illisible. Le traitement fonctionne correctement, c'est le cas qui n'est pas traitable automatiquement. Il se traite par un arrêt explicite et un aiguillage vers un humain, et il faut résister à la tentation de le camoufler en erreur technique.



Une seule règle : ne jamais réessayer un échec permanent, ne jamais alerter sur un échec transitoire résolu.

Figure 7.1 Trajectoires d'échec et réponse appropriée à chacune. Le mélange des trois familles produit les deux pathologies classiques : des alertes que plus personne ne lit, et des doublons créés par des tentatives sur des opérations non rejouables.

7.2 Quatre leviers

La nouvelle tentative s'active nœud par nœud, avec un nombre maximal d'essais et une attente entre eux. Elle ne s'active que sur les nœuds dont l'échec est plausiblement transitoire, et jamais sur un effet externe qui n'est pas rejouable, ce qui nous ramène à la section 7.4.

La poursuite sur échec laisse le workflow continuer malgré l'erreur. Sous sa forme moderne, la sortie d'erreur dirige les items en échec vers une branche distincte, ce qui vaut infiniment mieux que l'ancienne option de continuation : les items en échec sont identifiés, comptés, et traités séparément au lieu d'être noyés dans le flux principal.

L'arrêt explicite, par le nœud Stop And Error, sert à déclarer qu'un cas n'est pas traitable. Il produit une erreur portant un message que vous écrivez, ce qui vaut tous les débogages : « montant supérieur au plafond de validation automatique » est un message qui se lit dans une alerte, « TypeError undefined » n'en est pas un.

Le workflow d'erreur attrape ce qui reste, au niveau du workflow entier.

7.3 Le workflow d'erreur

Un workflow d'erreur est un workflow ordinaire commençant par un déclencheur d'erreur. On le désigne dans les réglages des workflows qu'il doit couvrir, et il reçoit alors le nom du workflow fautif, le nœud en cause, le message et l'identifiant d'exécution.

Il en faut un seul pour toute l'installation, pas un par workflow. Son travail est de router : notifier le bon canal, enregistrer une trace exploitable, et fournir le lien vers l'exécution pour permettre le rejeu. La tentation d'y mettre de la logique de rattrapage doit être combattue ; un workflow d'erreur qui échoue lui-même ne prévient personne.

RÉGLAGE

Nombre de tentatives et attente entre elles se règlent nœud par nœud, dans l'onglet des paramètres.

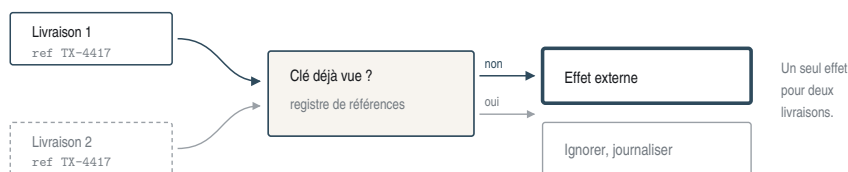
LA DERNIÈRE EXCUSE A DISPARU

Depuis le 1^{er} septembre 2026, les exécutions de workflows d'erreur ne sont plus décomptées du quota. L'argument du coût ne tient plus : tout workflow de production doit en avoir un.

7.4 Idempotence

Une opération est idempotente si l'exécuter deux fois avec les mêmes entrées produit le même état final qu'une seule fois. C'est la propriété qui rend la reprise possible, et sans elle toute la section précédente est dangereuse : réessayer une opération non idempotente, c'est créer un doublon.

Le rejeu n'est pas un cas rare. Un émetteur qui n'a pas reçu votre réponse à temps renvoie le même événement. Une nouvelle tentative rejoue un appel qui avait peut-être abouti sans que la réponse revienne. Un rejeu manuel après correction repasse dans le même chemin. Trois causes courantes, une seule protection.



La clé vient de l'émetteur.

Un horodatage de réception ou un identifiant tiré localement change à chaque rejeu et ne protège de rien : les deux livraisons paraîtraient distinctes.

Figure 7.2 Le contrôle d'idempotence, placé entre la réception et le premier effet externe. Sa valeur dépend entièrement du choix de la clé, et ce choix se fait au chapitre 4, sur le schéma, pas ici.

La plateforme offre un mécanisme prêt à l'emploi. Le nœud Remove Duplicates dispose d'une opération qui écarte les items déjà vus lors d'exécutions précédentes, avec un critère au choix : valeur nouvelle, valeur numérique supérieure à toute valeur antérieure, ou date postérieure. Il convient parfaitement aux flux de veille, aux collectes planifiées et aux compteurs incréments.

Il a une limite qu'il faut connaître avant de s'y fier pour de l'argent. La taille de l'historique est plafonnée, dix mille entrées par défaut, et le mécanisme d'éviction est une file : quand la limite est atteinte, les entrées les plus anciennes disparaissent. Un événement ancien qui reviendrait après cette rotation serait traité comme neuf. Pour un flux d'actualités, c'est sans conséquence. Pour des encaissements, ce n'en est pas une : la déduplication doit alors reposer sur une contrainte d'unicité dans une base externe, où le doublon est refusé par le stockage lui-même plutôt que par une vérification préalable qui peut être court-circuitée par deux exécutions simultanées.

LIMITE À CONNAÎTRE

Historique de dix mille entrées par défaut, en file : au-delà, les plus anciennes sortent et un vieil item repasse.

Deux remarques closent le sujet. La première est que l'ordre compte : le contrôle se place avant le premier effet externe et non après, sans quoi il ne protège rien. La seconde est que certaines API sont déjà idempotentes si on le leur demande, en acceptant une clé d'idempotence dans un en-tête. Quand cette possibilité existe, elle est supérieure à tout ce que vous construirez, parce que la protection s'applique du côté du service qui détient l'état.

7.5 Rejouer

Une exécution en échec reste consultable. La liste des exécutions se filtre par statut, par période et par données personnalisées enregistrées en cours de traitement, ce dernier point étant sous-utilisé : marquer une exécution avec la référence métier permet ensuite de retrouver le cas d'un client sans parcourir une liste chronologique.

Deux gestes suivent. Le jeu direct relance l'exécution telle quelle, ce qui convient quand la cause était transitoire. Le chargement des données de l'exécution dans l'éditeur permet de corriger le workflow puis de rejouer avec les données réelles qui l'avaient mis en défaut, et c'est la méthode de débogage la plus efficace de la plateforme.

Un jeu n'est légitime que si la section 7.4 a été traitée. Rejouer une exécution qui a déjà envoyé un message et créé une facture, sur un workflow non idempotent, produit un second message et une seconde facture. C'est la raison pour laquelle ce chapitre traite l'idempotence avant le jeu et non l'inverse.

7.6 Avant la mise en production

Sept points, à vérifier sur chaque workflow qui touche un système réel. Ils reviennent, complétés par les aspects d'exploitation, au chapitre 23.

- Un workflow d'erreur est désigné dans les réglages.
- Chaque effet externe a une clé d'idempotence identifiée, venue de l'émetteur.
- Les nouvelles tentatives sont activées sur les échecs transitoires uniquement.
- Les cas métier non traitables s'arrêtent avec un message lisible par un humain.
- Le compte d'items est vérifié juste avant chaque nœud qui écrit ou envoie.
- Les webhooks entrants sont vérifiés, par le nœud dédié ou à la main.
- Quelqu'un de nommé reçoit l'alerte, sur un canal qu'il consulte réellement.

CE QUE LA PARTIE III AJOUTE

Les deux premières parties ont traité la conception. La partie III traite l'exploitation : le déploiement en mode queue et son dimensionnement, la gestion des identités et des secrets, l'observabilité qui transforme une intuition en mesure, et le cycle de vie qui permet de modifier un parc sans le casser.

Déploiement : du Compose au mode queue

Une instance unique tient longtemps, puis cesse de tenir d'un coup. Ce chapitre décrit les trois paliers d'architecture, ce qui déclenche le passage de l'un à l'autre, et la poignée de réglages dont l'oubli fait échouer la bascule.

8.1 Trois paliers

Le premier palier est l'**instance unique**. Un seul processus fait tout : l'interface, l'API, les déclencheurs temporels, la réception des webhooks et l'exécution des workflows. C'est le mode par défaut, il convient parfaitement jusqu'à quelques dizaines d'exécutions concurrentes, et son défaut est structurel : si ce processus tombe, tout tombe avec lui.

Le deuxième est le **mode queue**. L'instance principale cesse d'exécuter et se contente d'orchestrer ; des processus ouvriers séparés font le travail, coordonnés par une file dans Redis. Une exécution qui plante n'emporte que son ouvrier, et on ajoute de la capacité en ajoutant des ouvriers plutôt qu'en achetant une machine plus grosse.

Le troisième est le **multi-main**, où plusieurs instances principales se partagent le rôle d'orchestrateur derrière un répartiteur de charge, avec une élection de meneur pour les déclencheurs temporels. Il supprime le dernier point unique de défaillance et relève de la licence Enterprise.

Le passage au deuxième palier ne se décide pas sur un taux d'occupation processeur. Il se décide sur un symptôme : une exécution longue qui retarde les autres, ou un redémarrage qui interrompt du travail en cours. Tant que ces deux phénomènes n'apparaissent pas, l'instance unique reste le bon choix, et sa simplicité vaut de l'argent.

LE SIGNAL

On passe au mode queue quand une exécution longue commence à retarder les autres, pas quand le processeur sature.

8.2 Ce qui change en mode queue

Le passage se fait par une variable d'environnement, et cette apparente simplicité masque quatre changements de fond.

```
EXECUTIONS_MODE=queue
QUEUE_BULL_REDIS_HOST=redis
QUEUE_BULL_REDIS_PORT=6379

# processus ouvrier
n8n worker --concurrency=10
```

D'abord, la **base de données devient critique**. Elle était un simple magasin, elle devient la source de vérité partagée : l'ouvrier y lit le workflow qu'on lui demande d'exécuter et y écrit le résultat. SQLite ne convient plus, et Postgres 13 ou supérieur est recommandé.

Ensuite, la **mémoire cesse d'être partagée**. Ce qui vivait dans le processus n'existe plus, ce qui explique la contrainte sur les données binaires traitée plus loin.

Puis, les **webhooks peuvent être servis séparément** par des processus dédiés placés derrière un répartiteur, ce qui isole la réception des pics de charge de l'exécution.

Enfin, et c'est le point qui fait échouer la moitié des bascules, **tous les processus doivent partager la même clé de chiffrement**.

LA CLÉ DE CHIFFREMENT, LE PIÈGE NUMÉRO UN

n8n génère automatiquement une clé au premier démarrage si vous n'en fournissez pas. Elle chiffre tous les identifiants stockés en base. Un ouvrier démarré sans la même valeur de `N8N_ENCRYPTION_KEY` que l'instance principale ne peut déchiffrer aucun identifiant : les exécutions échoueront toutes, avec des messages d'authentification qui n'ont aucun rapport apparent avec la cause. Cette clé doit être définie explicitement, identique partout, et sauvegardée séparément de la base : sans elle, les identifiants sont irrécupérables.

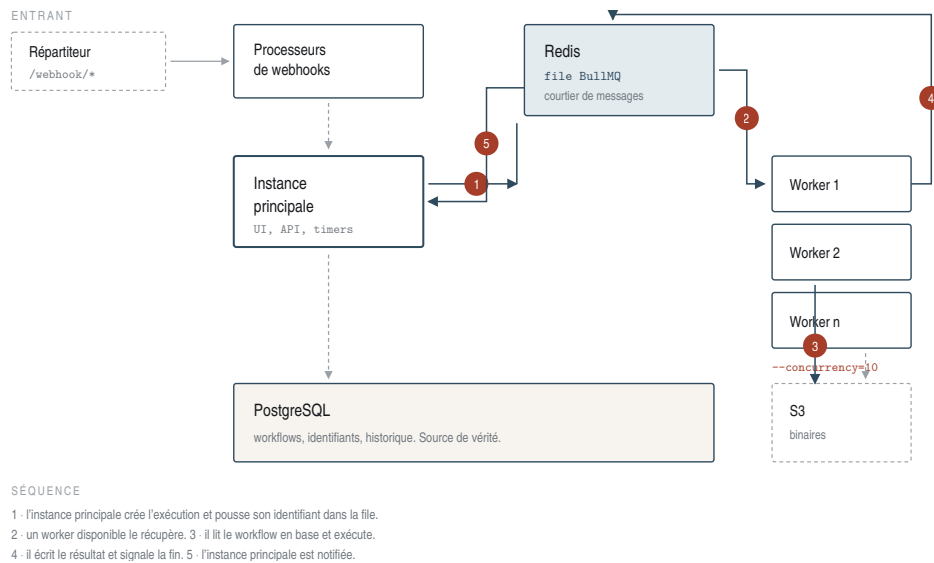


Figure 8.1 Architecture d'exécution en mode queue. Les traits pleins portent le chemin d'une exécution, les traits pointillés les dépendances permanentes. Chaque composant de ce schéma devient un point de configuration au chapitre 9, puis un point d'observation au chapitre 10.

8.3 Dimensionner

La concurrence d'un ouvrier se règle par un indicateur au démarrage et vaut dix par défaut. Une variable d'environnement peut la piloter globalement et prend alors le pas sur l'indicateur.

La tentation, devant une API fragile, est de baisser cette valeur très bas pour ménager le service distant. C'est une erreur : la documentation déconseille de descendre sous cinq, sous peine d'instabilité liée au pool de connexions à la base. Le cadencement d'un service distant se règle au niveau du nœud, comme vu au chapitre 6, pas au niveau de l'ouvrier.

Un point mérite d'être connu avant de compter sur le contrôle de concurrence : il ne s'applique qu'aux exécutions de production, celles issues d'un webhook ou d'un déclencheur. Les exécutions manuelles, les sous-workflows, les workflows d'erreur et les lancements en ligne de commande y échappent. Une file d'attente qui déborde de sous-workflows ne sera donc pas régulée par ce réglage.

8.4 Données binaires et réponses volumineuses

Par défaut, les fichiers manipulés par un workflow restent en mémoire. C'est rapide et cela fait tomber l'instance sur un fichier de deux cents mégaoctets. Le mode de stockage se change par une variable, avec quatre valeurs possibles : système de fichiers, base de données, S3 ou Azure.

Le mode système de fichiers **n'est pas supporté en mode queue**, puisque chaque ouvrier tourne sur sa propre machine et ne voit pas le disque des autres. Il faut donc choisir entre la base de données, qui fait transiter les fi-

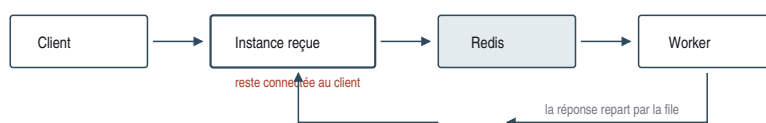
SEUIL

Ne pas descendre sous cinq. En dessous, le pool de connexions à la base se comporte mal et les performances s'effondrent.

chiers par votre base principale, et un stockage objet, qui est le seul choix raisonnable dès que les volumes comptent.

Le cas des réponses de webhook mérite un traitement à part parce qu'il surprend. En mode queue, l'ouvrier exécute mais le client reste connecté à l'instance qui a reçu la requête. La réponse produite par le nœud dédié doit donc revenir à cette instance, et elle voyage à l'intérieur d'un message de file : Redis détient la totalité de la réponse pendant le trajet. Une limite plafonne ce message, à soixante-quatre mébiotets par défaut, et au-delà le nœud échoue. Depuis la version 2.34, un déchargement peut être activé : le corps part alors dans le stockage binaire et seule une référence circule.

CHEMIN DE LA RÉPONSE



AU-DELÀ DE LA LIMITE



Figure 8.2 Chemin de la réponse d'un webhook en mode queue. Le détail qui surprend est que la réponse traverse Redis : ce n'est pas l'ouvrier qui parle au client, mais l'instance qui a reçu la requête, et le corps doit donc faire le trajet inverse dans un message de file.

8.5 Purger

Une installation qui n'efface rien finit par avoir une base plus grosse que tout ce qu'elle a jamais traité. L'historique des exécutions se purge selon trois critères combinables : un âge maximal, un nombre maximal conservé, et une fréquence de nettoyage. La purge des données binaires suit celle des exécutions, avec une subtilité qui piège : elle n'opère que sur le mode de stockage actif. Un passage du système de fichiers vers S3 laisse derrière lui tout l'ancien contenu, indéfiniment, sur un disque que plus personne ne regarde.

Le réglage de la fenêtre de rétention n'est pas une question technique mais une question métier. Elle doit couvrir le délai au bout duquel un incident est encore susceptible d'être investigué, et le chapitre 10 explique pourquoi une rétention plus courte que la fenêtre d'analyse produit des tableaux de bord vides qu'on prend pour des pannes.

8.6 Multi-main

Deux conditions gouvernent ce palier. Toutes les instances principales doivent avoir le mode multi-main activé, et elles doivent se trouver derrière un répartiteur avec persistance de session. Un meneur est élu pour porter les déclen-

cheurs temporels, avec une durée de vie de clé et un intervalle de vérification réglables.

La persistance de session n'est pas un détail d'optimisation : sans elle, la deuxième requête d'un échange peut atterrir sur une instance qui ignore tout de la première. Le chapitre 13 rencontrera le même problème avec les connexions durables des serveurs MCP, qui doivent être routées vers une réplique dédiée.

Sécurité, identités et secrets

Une instance n8n est un agrégateur d'identifiants. Vingt workflows peuvent y concentrer les accès au système de paiement, au CRM, à la base de production et à la messagerie. Compromettre l'instance revient à compromettre tout cela d'un coup.

9.1 Ce que l'instance concentre

Le raisonnement de sécurité doit partir de là, et pas de la liste des fonctionnalités disponibles. Chaque connexion à un service extérieur laisse dans la base un jeton, une clé ou une chaîne de connexion. Ces éléments sont chiffrés, mais ils sont réunis, et cette réunion crée une valeur qui n'existait nulle part avant l'installation.

Trois conséquences pratiques en découlent. L'interface d'administration ne doit jamais être exposée directement sur Internet, quel que soit le mot de passe. Les identifiants doivent être portés par les mécanismes prévus plutôt que copiés dans des paramètres de nœuds. Et l'accès des personnes doit être révoquant en un geste, ce qui plaide pour une authentification déléguée.

9.2 La clé de chiffrement

Le chapitre précédent l'a présentée comme un piège de déploiement. C'est aussi l'objet de sécurité le plus important de l'installation, et il mérite ses propres règles.

Elle doit être définie explicitement plutôt que laissée à la génération automatique, être identique sur tous les processus, et être sauvegardée séparément de la base de données. Ce dernier point est celui qu'on néglige : une sauvegarde contenant à la fois la base et la clé annule le bénéfice du chiffrement, tandis qu'une clé perdue rend tous les identifiants définitivement illisibles, sans mécanisme de récupération.

SAUVEGARDE

La clé se sauvegarde ailleurs que la base. Ensemble au même endroit, elles ne protègent de rien.

La rotation est possible mais elle n'est pas transparente : changer la clé n'a pas d'effet rétroactif sur ce qui a déjà été chiffré. La procédure consiste à ressaisir les identifiants après le changement, ce qui suppose de les avoir par ailleurs. C'est une raison de plus pour que les secrets vivent dans un coffre externe plutôt que seulement dans n8n.

9.3 Identités et droits

Le modèle repose sur trois niveaux. Les **utilisateurs** ont un rôle au niveau de l'instance. Les **projets** regroupent workflows, identifiants et tables de données, et servent d'unité d'isolation entre équipes ou entre clients. Les **rôles** définissent ce qu'on peut faire à l'intérieur d'un projet.

Depuis la version 2.2, ces rôles peuvent être personnalisés, et la séparation qu'ils rendent possible est celle qui compte réellement en production : le droit de modifier un workflow, le droit de le publier et le droit de l'exécuter sont trois choses distinctes. Un intégrateur peut avoir le premier sans le deuxième ; un opérateur peut avoir le troisième sans les autres. Cette distinction est l'équivalent, dans le monde des workflows, de la séparation entre l'écriture du code et le déploiement en production.

L'authentification déléguée, par OIDC ou SAML, centralise les politiques de mot de passe et l'authentification à deux facteurs, mais son principal apport est ailleurs : désactiver quelqu'un dans l'annuaire lui retire immédiatement l'accès à n8n. Sans elle, le départ d'un collaborateur laisse un compte actif que personne ne pense à fermer.

9.4 Les secrets externes

Un coffre externe déplace la valeur sensible hors de n8n : l'identifiant enregistré ne contient plus la clé mais une référence, et la valeur est résolue au moment de l'exécution. Vault, AWS, Azure, GCP et, depuis la version 2.12, 1Password sont pris en charge.

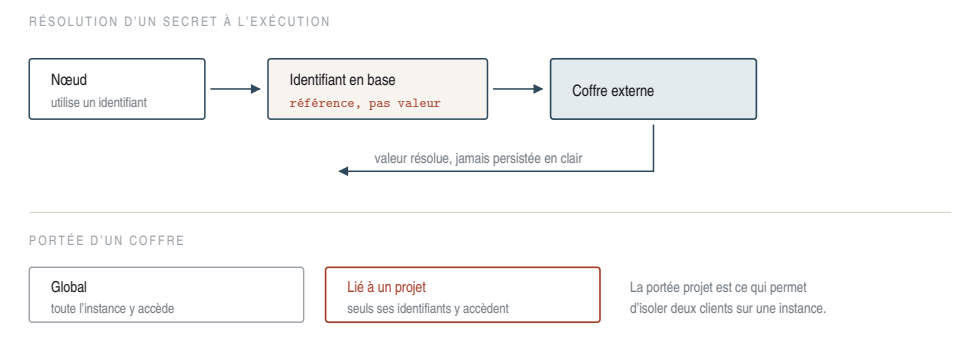


Figure 9.1 Résolution d'un secret externe. Le gain n'est pas seulement le chiffrement, que n8n assure déjà : c'est la centralisation de la rotation et la possibilité de donner à chaque environnement des valeurs différentes sous une même référence.

Un coffre est global par défaut, ce qui signifie que tous les identifiants de l'instance peuvent y faire référence. Un administrateur peut le lier à un projet, et seuls les identifiants de ce projet y accèdent alors. C'est ce mécanisme, et non la séparation des workflows, qui permet d'héberger deux clients sur une même installation sans que l'un puisse atteindre les secrets de l'autre.

Une subtilité vaut d'être connue avant de promettre une isolation forte. La configuration par détection automatique des identifiants d'exécution, pratique parce qu'elle évite de gérer une clé d'accès de longue durée, résout l'identité disponible dans l'environnement d'exécution. Tous les coffres ainsi configurés sur la même instance partagent donc cette identité unique, et il devient impossible de leur donner des permissions différentes. Quand l'isolation compte, il faut des identités explicites.

DÉTAIL AWS

Tous les coffres configurés en détection automatique sur une même instance partagent une seule identité et un seul jeu de permissions.

9.5 Ce que la version 2.0 a fermé

Plusieurs surfaces d'attaque ont été refermées, et connaître ces changements évite de chercher longtemps pourquoi un workflow hérité ne fonctionne plus.

- Le nœud Code s'exécute dans un environnement isolé et n'accède plus aux variables d'environnement de l'instance. Un workflow qui lisait une clé de cette façon doit passer par un identifiant ou un coffre.
- Les nœuds d'exécution de commande système et de surveillance de fichiers locaux sont désactivés par défaut. Ils se réactivent explicitement, et ce geste doit être documenté.
- L'occultation des données d'exécution, apparue en 2.16, permet d'empêcher que des valeurs sensibles se retrouvent visibles dans l'historique consultable par toute l'équipe.

9.6 Ce qu'il faut vérifier

Huit points, à passer une fois par installation puis à chaque changement d'infrastructure.

- La clé de chiffrement est explicite, identique sur tous les processus, et sauvegardée ailleurs que la base.
- L'interface n'est joignable que derrière un accès contrôlé, jamais directement exposée.
- L'authentification est déléguée à l'annuaire, et la désactivation d'un compte y produit un effet immédiat.
- Les rôles séparent l'édition, la publication et l'exécution.
- Chaque client ou équipe a son projet, et les coffres sensibles sont liés à ces projets.
- Aucun secret ne figure en clair dans un paramètre de nœud.

- Les nœuds désactivés par défaut le sont restés, ou leur réactivation est justifiée par écrit.
- Les webhooks entrants sont vérifiés, comme vu au chapitre 6.

Observabilité et preuve de valeur

C'est le chapitre pivot des deux pistes de lecture. Le praticien le lit pour savoir ce qui se passe dans son installation ; le décideur pour savoir ce qu'il doit exiger avant de financer la suivante.

10.1 Trois questions différentes

On confond régulièrement trois interrogations qui n'appellent ni les mêmes outils ni les mêmes destinataires.

Est-ce que ça tourne ? C'est la question de la disponibilité, et elle se répond par la supervision d'infrastructure ordinaire : les processus sont-ils vivants, la base répond-elle, la file se vide-t-elle. Rien de spécifique à n8n.

Est-ce que ça marche ? C'est la question de la justesse : les exécutions aboutissent-elles, où échouent-elles, combien de temps prennent-elles, et pourquoi telle branche est-elle plus lente depuis mardi. C'est le domaine des traces.

Est-ce que ça rapporte ? C'est la question du décideur, et elle est le plus souvent sans réponse parce que personne n'a instrumenté ce qu'il fallait au moment où il fallait. C'est le domaine des indicateurs métier.

Une installation professionnelle répond aux trois. La plupart des installations ne répondent qu'à la première, ce qui explique pourquoi tant de projets d'automatisation ne trouvent pas de second budget.

10.2 Les traces

Depuis la version 2.15, n8n émet nativement des traces au format OpenTelemetry, et cette fonction change la nature des conversations avec une équipe d'exploitation : elle fait entrer les workflows dans l'outillage que l'entreprise possède déjà.

Le modèle est simple. Chaque exécution produit une portée `workflow.execute`, qui porte l'identifiant et le nom du workflow, sa version, le nombre de nœuds,

DISPONIBILITÉ

La télémétrie se configure par variables d'environnement : elle n'existe donc qu'en auto-hébergement.

le mode d'exécution et le statut final. À l'intérieur, chaque nœud produit une portée `node.execute` imbriquée, qui porte son nom, son type, et surtout le nombre d'items reçus et produits.

Ce dernier détail mérite qu'on s'y arrête, parce qu'il transforme en mesure quelque chose que le chapitre 3 présentait comme une discipline de lecture. Le nombre d'items entrants et sortants de chaque nœud, que l'on comptait à l'œil sur le canevas, devient un attribut interrogeable et historisable. Un filtre qui laisse soudain passer trois cents items au lieu de trois se voit dans une courbe, avant que la facture ne le signale.

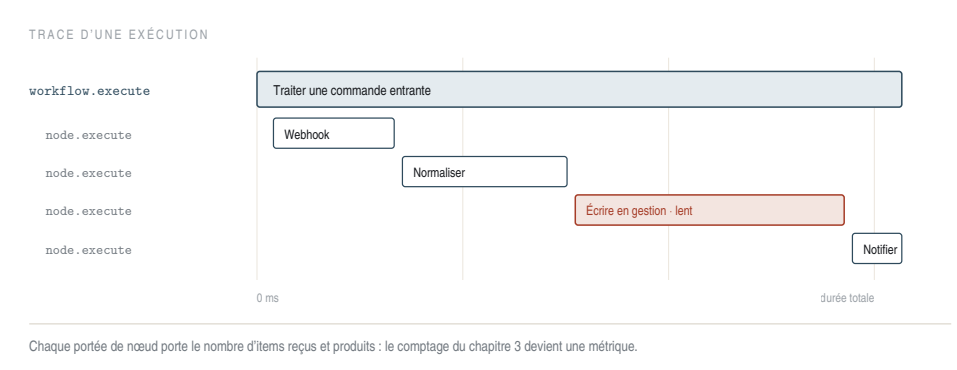


Figure 10.1 Portées imbriquées d'une exécution. Le coupable apparaît sans ouvrir l'éditeur, et le contexte de trace se propageant dans les deux sens, un appel sortant lent se relie à la requête entrante qui l'a provoqué.

Deux compléments méritent d'être connus. Le contexte de trace se propage dans les deux sens, ce qui permet de relier une exécution n8n à la requête applicative qui l'a déclenchée et aux appels qu'elle a émis. Et une exécution rejouée est marquée comme telle, avec la référence de l'exécution d'origine, ce qui évite de compter deux fois le même incident dans les statistiques.

10.3 Les journaux

La diffusion de journaux envoie les événements d'audit et de cycle de vie vers un collecteur extérieur. Elle répond à une question différente de celle des traces : non pas comment s'est déroulée telle exécution, mais qui a fait quoi sur l'installation. Création d'identifiant, publication d'un workflow, modification d'un rôle, connexion d'un utilisateur.

Cette distinction compte au moment d'un incident. Une trace explique une panne technique ; un journal d'audit explique un changement de comportement dont personne ne se souvient. Les deux sont nécessaires et se rangent dans des outils différents.

10.4 Insights et temps gagné

Le tableau de bord Insights agrège les exécutions par workflow : volume, taux d'échec, durée. C'est l'outil de pilotage courant, celui qu'on ouvre le lundi matin.

Le nœud Time Saved est plus intéressant et beaucoup moins utilisé. Il se place sur une branche et déclare le temps que cette branche a évité de consommer. La valeur ainsi produite s'agrège dans les Insights et devient le chiffre du chapitre 1, mais mesuré au lieu d'être estimé.

Sa vertu est d'être placé *sur une branche*, ce qui le rend fidèle. Un dossier traité entièrement de façon automatique n'économise pas le même temps qu'un dossier passé par une validation humaine, et un dossier tombé dans la branche d'exception n'économise rien du tout. Placer un nœud par branche, avec des valeurs différentes, produit un chiffre qui résiste à la contestation, là où une estimation globale multipliée par le volume ne résiste à aucune.

Une précaution s'impose sur la rétention. La fenêtre d'analyse ne peut pas dépasser la durée pendant laquelle les données sont conservées, et une plage de dates plus large que la rétention configurée produit un graphique vide que l'on prend pour une panne. La rétention des indicateurs se règle indépendamment de celle des exécutions et peut aller jusqu'à un ou deux ans selon la licence : c'est ce réglage, et non la purge des exécutions, qui détermine jusqu'où remonte le rapport annuel.

10.5 Alerter sans lasser

Une alerte que plus personne ne lit est pire qu'aucune alerte, parce qu'elle donne l'illusion d'une surveillance. Trois règles suffisent à l'éviter.

N'alerter que sur ce qui appelle une action. Un échec transitoire résolu par une nouvelle tentative ne mérite pas de notification ; il mérite d'apparaître dans une statistique hebdomadaire.

Alerter avec ce qu'il faut pour agir : le nom du workflow, le nœud, le message, et le lien direct vers l'exécution. Une notification qui oblige à chercher est une notification qu'on remet à plus tard.

Alerter quelqu'un de nommé, sur un canal qu'il consulte. Une adresse de groupe est un endroit où les alertes vont mourir.

10.6 Le rapport mensuel

Quatre chiffres suffisent, et ils tiennent sur une demi-page. Le volume traité, le taux de couverture réel, le temps gagné mesuré, et la liste des incidents avec leur cause.

Le troisième chiffre est celui qui finance la suite. Le quatrième est celui qui achète la confiance, à condition d'être publié même quand il est mauvais : un prestataire qui signale ses propres incidents avant qu'on les découvre obtient des budgets que ses concurrents n'obtiennent pas.

FAUX PROBLÈME CLASSIQUE

Un graphique vide sur trois mois quand la rétention en couvre trente jours. Ce n'est pas une panne, c'est une purge.

Cycle de vie : versions, environnements, CI/CD

Modifier un workflow qui tourne est un acte de production. La plateforme fournit depuis la version 2.0 les outils pour le traiter comme tel ; encore faut-il les employer.

11.1 Enregistrer n'est plus publier

Le changement le plus structurant de la version 2.0 tient en une phrase, déjà annoncée au chapitre 3 : l'enregistrement et la publication sont deux gestes distincts. On travaille sur une version enregistrée, et la version publiée continue de tourner inchangée jusqu'à ce qu'on décide de la remplacer.

C'est un progrès considérable et une source de confusion pendant quelques semaines. La confusion produit toujours le même symptôme : un correctif appliqué et enregistré, un comportement qui ne change pas, et une demi-heure passée à chercher l'erreur ailleurs. La règle mnémotechnique est brutale et efficace : tant que vous n'avez pas publié, vous n'avez rien changé.

11.2 Versions et comparaison

L'historique conserve les états successifs d'un workflow et permet d'y revenir. Depuis la version 2.13, la comparaison entre deux versions est visuelle : au lieu de lire un JSON, on voit sur le canevas ce qui a été ajouté, retiré ou modifié.

L'usage principal n'est pas le retour en arrière, qui reste rare, mais le diagnostic. Devant un workflow qui s'est mis à échouer, la première question est de savoir ce qui a changé depuis la dernière exécution réussie, et la comparaison visuelle y répond en quelques secondes.

USAGE RÉEL

Le premier réflexe devant une régression n'est pas de déboguer, c'est de comparer avec la version d'avant.

11.3 Contrôle de source et environnements

La fonction de contrôle de source adosse les workflows à un dépôt Git. Chaque instance est reliée à une branche, et les changements se poussent

d'une instance vers le dépôt puis se tirent vers une autre. C'est ainsi que se construit un pipeline de promotion entre un environnement de développement, un environnement de recette et la production.

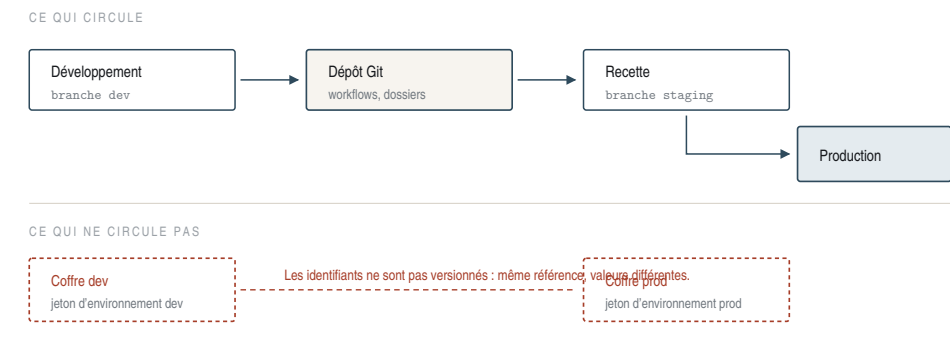


Figure 11.1 Promotion d'un workflow entre environnements. Le trait plein porte ce que Git transporte ; le trait pointillé rappelle ce qu'il ne transporte pas, et qui doit donc être résolu autrement à chaque étage.

11.4 Le problème des identifiants

Voici la contrainte que l'on découvre au premier essai, et qui décide de la faisabilité de tout le dispositif : le contrôle de source ne gère pas des identifiants différents selon l'instance. Un workflow promu en production arriverait donc avec les accès de développement, ce qui est inacceptable et, la plupart du temps, découvert au pire moment.

La solution officielle passe par les coffres externes du chapitre précédent, et elle est élégante. Chaque instance se connecte à un environnement différent du même fournisseur de secrets, avec son propre jeton. Le workflow, lui, ne contient qu'une référence, identique partout. La même référence résout la valeur de développement sur l'instance de développement et la valeur de production sur celle de production, sans qu'aucune modification du workflow ne soit nécessaire.

Cette dépendance mérite d'être posée clairement, parce qu'elle change l'ordre des chantiers : un pipeline de promotion propre suppose un coffre externe. Vouloir le premier sans le second conduit à des bricolages qui finissent tous par un identifiant de production recopié à la main.

11.5 Paquets et API

Deux autres mécanismes complètent le tableau, avec des usages distincts.

Les **paquets** exportent un ensemble cohérent de workflows dans un fichier transportable. Ils conviennent à la livraison ponctuelle vers une installation qui n'est pas reliée à votre dépôt, cas fréquent chez un client qui héberge lui-même.

L'**API publique** permet d'automatiser ce qui précède : lister, créer, activer, désactiver des workflows, gérer des identifiants, interroger les indicateurs.

C'est par elle que passent les scripts de déploiement, les contrôles de conformité et les inventaires. Sur un parc de plusieurs dizaines de workflows, un inventaire automatisé est le seul moyen de savoir ce qui tourne réellement.

11.6 Une discipline minimale

Tout ce qui précède peut sembler lourd pour une installation de dix workflows. Voici la version réduite, qui tient en cinq habitudes et ne coûte presque rien.

- Publier est un geste distinct, annoncé, et jamais fait un vendredi soir.
- Avant de modifier un workflow qui tourne, on regarde sa dernière exécution réussie.
- Devant une régression, on compare les versions avant de déboguer.
- Les secrets vivent dans un coffre, même avec un seul environnement, parce que c'est ce qui rendra le deuxième possible.
- Un inventaire des workflows actifs est produit et relu tous les mois.

CE QUE LA PARTIE IV AJOUTE

Les trois premières parties décrivent des workflows déterministes : à entrée donnée, sortie prévisible. La partie IV introduit des composants qui ne le sont pas. Elle traite d'abord les canaux conversationnels et leur économie, puis les agents et leurs outils, la connaissance métier interrogeable, l'orchestration multi-agents, et enfin les garde-fous sans lesquels rien de tout cela n'a sa place en production.

Canaux conversationnels : chat web et WhatsApp

Un canal conversationnel n'est pas un déclencheur de plus. Il introduit trois choses que les onze chapitres précédents n'ont pas rencontrées : un état qui dure au-delà d'une exécution, un interlocuteur qui attend, et une facturation au message.

12.1 Ce que la conversation change

Un webhook reçoit un événement complet et autonome. Un message de conversation, lui, n'a de sens que par rapport à ce qui a été dit avant. Trois conséquences en découlent, et elles gouvernent tout ce chapitre.

L'état survit à l'exécution. Le chapitre 3 rappelait qu'une exécution ne sait rien des précédentes. Une conversation exige exactement l'inverse, et cet état doit donc être porté par un mécanisme explicite.

Quelqu'un attend. Le confort d'un traitement qui prend douze secondes disparaît quand un humain regarde son écran. C'est ce qui rend la diffusion en flux, traitée au chapitre 13, plus qu'un raffinement.

Chaque réponse a un prix. Sur le canal web, le prix est celui du modèle. Sur WhatsApp, il s'y ajoute un coût par message dont les règles changent le 1^{er} octobre 2026, et cette bascule justifie à elle seule la moitié du chapitre.

12.2 Le déclencheur de chat

Le déclencheur de chat expose une interface de conversation. Deux modes existent, et le choix entre eux a des conséquences techniques qui vont bien au-delà de l'apparence.

Le mode **hébergé** fournit une page de chat prête à l'emploi, servie par n8n. Il convient à un usage interne, à une démonstration ou à un outil d'équipe. Le mode **embarqué** fournit un composant à intégrer dans votre propre site, avec les réglages d'origines autorisées et d'authentification qui vont avec.

Le piège se trouve dans la façon de répondre. Le nœud Chat, pratique pour renvoyer un message et attendre une réponse, n'est pas supporté en mode embarqué : il faut passer par le nœud de réponse au webhook. Il ne fonctionne pas davantage à l'intérieur d'un sous-workflow ni comme outil d'un sous-agent. Une architecture qui découpe proprement selon les règles du chapitre 4 se heurte donc à cette limite, et il vaut mieux la connaître avant de dessiner que pendant l'intégration.

PIÈGE

Le nœud Chat n'est pas supporté en mode embarqué. Il ne fonctionne pas non plus dans un sous-workflow, ni comme outil d'un sous-agent.

12.3 L'identifiant de session

C'est la clé qui relie les messages d'une même conversation, et c'est le paramètre le plus lourd de conséquences de tout le dispositif conversationnel.

Sur le canal web, il est fourni par le composant. Sur WhatsApp, il découle naturellement du numéro de l'interlocuteur. Dans les deux cas, trois questions se posent et méritent une réponse écrite avant l'implémentation. Combien de temps une session doit-elle vivre ? Que se passe-t-il quand deux personnes écrivent depuis le même poste ? Et surtout, qu'est-ce qui reste accessible dans l'historique d'une session à quelqu'un qui reprendrait la même clé plus tard ?

Cette dernière question n'est pas théorique. Une clé de session prévisible, dérivée d'un identifiant devinable, permet de récupérer la conversation d'un autre. C'est le principal risque de sécurité propre aux canaux conversationnels, et il ne ressemble à aucun de ceux du chapitre 9.

12.4 WhatsApp : le nœud et son déclencheur

Le nœud WhatsApp Business Cloud couvre l'envoi d'un message, l'envoi avec attente de réponse, l'envoi d'un gabarit, ainsi que l'envoi, la récupération et la suppression de médias. Le nœud fonctionne avec une clé d'API ; son déclencheur, lui, exige une authentification OAuth2, ce qui surprend et fait perdre du temps à qui n'a préparé qu'un seul type d'identifiant.

LE PIÈGE QUI FAIT ÉCHOUER LES MISES EN PRODUCTION

WhatsApp n'autorise qu'un seul webhook par application. Chaque bascule entre l'adresse de test et l'adresse de production écrase la précédente. Concrètement, ouvrir l'éditeur pour tester coupe la production, et personne ne s'en aperçoit avant que les messages cessent d'arriver. La parade est d'utiliser deux applications Meta distinctes, une par environnement, et de le prévoir dès la conception plutôt qu'après le premier incident.

Un point de calendrier mérite d'être placé au plan de charge dès le premier jour : la vérification du compte Meta Business prend une à deux semaines. Elle se lance en parallèle du développement, sans quoi elle devient le chemin critique d'un projet par ailleurs terminé.

12.5 L'économie du canal

Voici la partie que les tutoriels ignorent et qui détermine si le cas d'usage est rentable. Le modèle a changé deux fois en dix-huit mois et change à nouveau le mois prochain.

Le principe stable est celui de la **fenêtre de service**. Quand un client vous écrit, une fenêtre de vingt-quatre heures s'ouvre, pendant laquelle vous pouvez répondre librement, sans gabarit pré-approuvé. Chaque nouveau message du client la réinitialise. En dehors de cette fenêtre, seuls des gabarits approuvés peuvent partir. Les messages entrants, eux, restent gratuits quel qu'en soit le nombre.

Ce qui change, c'est le prix de ce qui sort dans la fenêtre. Depuis novembre 2024, les réponses libres y étaient gratuites ; depuis juillet 2025, les gabarits utilitaires l'étaient aussi. **Le 1^{er} octobre 2026, ces deux gratuités disparaissent.** Les réponses libres deviennent facturées au message, au tarif utilitaire du pays du destinataire, et les gabarits utilitaires envoyés dans la fenêtre le redeviennent également. Les paliers de volume dont bénéficient les gabarits ne s'appliquent pas aux réponses libres.

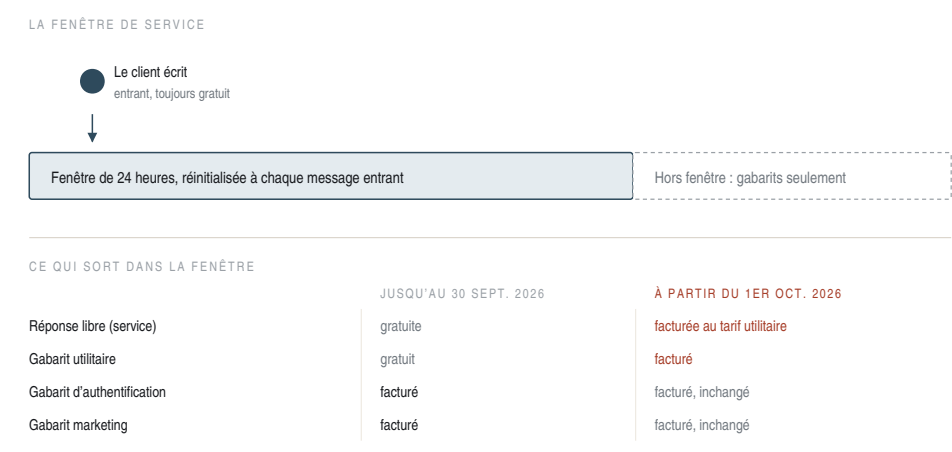


Figure 12.1 Facturation des messages sortants selon leur catégorie et leur position par rapport à la fenêtre de service.

Les deux lignes du haut basculent le 1^{er} octobre 2026. Les entrants restent gratuits, et la fenêtre continue de gouverner ce qu'on a le droit d'envoyer sans gabarit.

Une franchise mensuelle de réponses libres par numéro d'entreprise est annoncée, ainsi qu'une facturation à jeton pour l'agent conversationnel intégré de Meta depuis le 1^{er} août 2026. Ces montants sont susceptibles d'évoluer et doivent être relus sur la documentation de l'éditeur au moment de chiffrer : ce livre donne la structure du modèle, pas la grille tarifaire.

Un détail technique permet de vérifier soi-même plutôt que de croire un article. La notification de statut renvoyée par l'API porte un type de tarification. Une réponse libre gratuite y apparaît aujourd'hui sous un libellé de service client ; facturée, elle apparaîtra sous le libellé ordinaire. Journaliser ce champ

À VÉRIFIER

Les tarifs par pays et le détail des franchises sont publiés par Meta. Toute étude de coût doit être refaite à la date du projet.

dans le workflow donne un compteur de coût exact, propre à votre trafic, et détecte la bascule le jour où elle survient.

12.6 Le plafond de fréquence

Indépendamment du prix, un plafond limite le nombre de messages marketing qu'un même utilisateur peut recevoir par tranche de vingt-quatre heures, tous émetteurs confondus. L'API refuse les envois au-delà avec un code d'erreur dédié.

Deux conséquences pratiques. La première est qu'aucun contournement ne fonctionne : le plafond porte sur le destinataire, pas sur l'émetteur, et multiplier les numéros ou changer de fournisseur ne change rien. La seconde est qu'un refus de ce type n'est pas un échec technique au sens du chapitre 7 : c'est un échec métier, il ne faut pas le réessayer, et le workflow doit le traiter comme tel.

12.7 Ce que l'économie impose à la conception

Quatre règles découlent directement de ce qui précède, et elles se décident au moment du schéma.

- **Privilégier les flux entrants.** Un canal où le client écrit en premier coûte structurellement moins cher qu'un canal où l'on relance. Cela oriente le cas d'usage vers le support et la réponse, plutôt que vers la prospection.
- **Regrouper les réponses.** Quand chaque message sortant se paie, trois messages successifs coûtent trois fois une réponse unique bien construite. La conception conversationnelle devient une question de coût, pas seulement de style.
- **Traiter la catégorie comme une donnée métier.** La frontière entre utilitaire et marketing se joue sur l'intention promotionnelle et se décide à l'approbation du gabarit. Une confirmation d'expédition est utilitaire ; la même assortie d'un code de réduction devient marketing, avec le tarif correspondant.
- **Mesurer dès le premier jour.** Le champ de tarification journalisé donne le coût réel par conversation, seul chiffre qui permette de dire si le canal est rentable.

De la règle à l'agent

Un workflow classique décide à votre place selon des règles que vous avez écrites. Un agent décide selon des règles qu'il formule lui-même, à chaque exécution. C'est un gain de souplesse et une perte de déterminisme, et tout l'art consiste à savoir où placer la frontière.

13.1 Ce qu'un agent fait réellement

Le mot recouvre une mécanique simple, qu'il vaut mieux se représenter précisément pour ne pas lui prêter des pouvoirs qu'elle n'a pas.

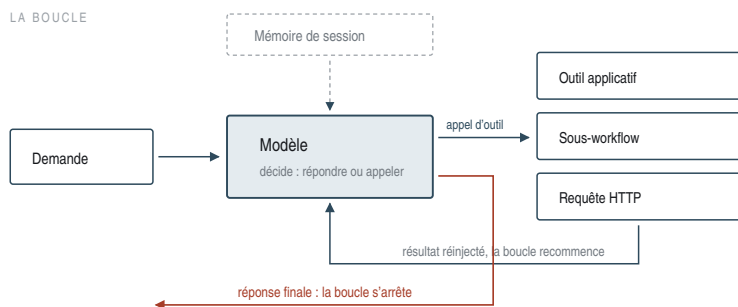
Un agent est une boucle. Le modèle reçoit une demande, la liste des outils dont il dispose avec leur description, et l'historique de la conversation. Il répond soit par un texte, ce qui termine la boucle, soit par une demande d'appel d'outil. Dans le second cas, n8n exécute l'outil, renvoie le résultat au modèle, et la boucle recommence.

Deux choses découlent de cette mécanique. D'abord, la **description de chaque outil est du code** : c'est le seul élément sur lequel le modèle s'appuie pour décider s'il doit l'appeler. Une description vague produit un agent qui n'appelle jamais l'outil, ou qui l'appelle toujours. Ensuite, le **nombre de tours n'est pas borné par votre workflow** mais par la décision du modèle, ce qui rend le coût et la durée d'une exécution variables par nature.

CONSÉQUENCE

La description d'un outil est un paramètre fonctionnel, pas un commentaire. C'est sur elle que le modèle décide.

LA BOUCLE



Le nombre de tours est décidé par le modèle : durée et coût d'une exécution sont variables par construction.

Figure 13.1 Boucle d'exécution d'un agent. Les outils sont branchés en sous-nœuds sur le nœud racine, ce qui en fait un nœud en grappe au sens du chapitre 3, avec l'asymétrie de résolution d'expression que cela implique.

13.2 Un seul type d'agent

Les versions antérieures proposaient plusieurs modes d'agent : conversationnel, ReAct, planification, interrogation SQL. Ce réglage a disparu, et tous les agents fonctionnent désormais selon le mécanisme d'appel d'outils décrit ci-dessus. La version 3.0 retire définitivement l'ancienne version du nœud.

C'est une simplification bienvenue et un point de vigilance pour les workflows hérités : un agent configuré sur un ancien mode doit être reconstruit, et le comportement obtenu n'est pas strictement identique. Le chapitre 23 donne la méthode de repérage sur un parc existant.

13.3 Les outils

Quatre familles d'outils couvrent tous les besoins.

Les **outils applicatifs** exposent directement une opération d'un service connu. Les **requêtes HTTP** couvrent le reste. L'**appel de sous-workflow** est le plus intéressant des quatre, parce qu'il permet d'exposer à l'agent une logique métier construite selon les règles des parties précédentes, avec ses contrôles, son idempotence et son workflow d'erreur. Enfin, l'**outil de code** convient aux calculs purs.

La recommandation qui découle du reste de ce livre est nette : un agent ne devrait presque jamais appeler directement une API qui écrit. Il devrait appeler un sous-workflow qui, lui, écrit, avec toute la protection du chapitre 7. Cela déplace la garantie du côté déterministe, là où on sait la construire, et laisse à l'agent le seul rôle qu'il tient bien, celui de choisir quoi appeler.

13.4 Les paramètres remplis par le modèle

Un outil a besoin de paramètres, et ces paramètres viennent souvent de la conversation plutôt que du workflow. Un assistant qui interroge une commande a besoin du numéro de commande, que seul le client connaît.

Le mécanisme prévu laisse le modèle remplir un paramètre, à partir d'une description que vous rédigez. Cette description est, là encore, du code fonctionnel : elle dit au modèle quoi extraire et sous quelle forme. Une description qui précise le format attendu réduit considérablement le taux d'appels invalides.

Une précaution de sécurité s'impose et sera reprise au chapitre 16. Un paramètre rempli par le modèle est, indirectement, un paramètre rempli par l'utilisateur qui parle au modèle. Tout ce que le chapitre 6 disait de la vérification des entrées s'applique, et rien de ce qui vient d'un agent ne doit atteindre une base de données ou une commande sans validation.

13.5 La mémoire

La mémoire est un sous-nœud qui conserve l'historique d'une conversation, indexé par l'identifiant de session du chapitre précédent. Deux choix se présentent.

La **mémoire simple** vit dans l'instance. Elle convient au prototype et disparaît au redémarrage, ce qui la rend inadaptée à un service réel, et franchement dangereuse en mode queue où plusieurs ouvriers ne partagent rien. La **mémoire en base**, typiquement Postgres, survit et se partage entre processus. C'est le seul choix raisonnable en production.

Un réglage compte plus que le choix du support : la longueur de la fenêtre d'historique transmise au modèle. Trop courte, l'agent oublie ce qui vient d'être dit. Trop longue, chaque tour coûte davantage et la latence augmente, sans gain de pertinence. C'est un paramètre à régler par l'observation, pas par principe.

RÈGLE

Mémoire simple pour prototyper, mémoire en base dès que le service doit survivre à un redémarrage.

13.6 La diffusion en flux

Sur un canal conversationnel, la réponse peut s'afficher au fur et à mesure de sa production plutôt qu'en bloc à la fin. Ce comportement est actif par défaut et suppose un déclencheur de chat, ou un webhook réglé en mode de réponse continue.

L'apport n'est pas cosmétique. Une réponse qui commence à s'afficher après une seconde est perçue comme rapide même si elle met huit secondes à se terminer, tandis qu'un silence de huit secondes est perçu comme une panne. Sur un canal où quelqu'un attend, la perception de la latence est un critère de conception au même titre que la latence elle-même.

13.7 Quand ne pas mettre d'agent

La question mérite d'être posée à chaque fois, parce que la réponse est souvent négative et que le coût d'un agent inutile est double : de l'argent par tour, et du non-déterminisme dans un endroit qui n'en avait pas besoin.

- **Si la règle est écrite**, un nœud de condition la fait mieux, plus vite et gratuitement. Un agent qui choisit entre deux branches selon un montant est un gaspillage.
- **Si l'entrée est structurée**, l'extraction n'a pas besoin d'un agent : un nœud d'extraction structurée suffit et reste prévisible.
- **Si l'opération est irréversible et sans contrôle humain**, l'agent n'a pas sa place, quel que soit le soin apporté aux instructions.
- **Si le volume est élevé**, le coût par tour se multiplie par le trafic, et l'arithmétique décide à votre place.

Reste le terrain où l'agent est difficile à remplacer : une entrée en langage naturel, imprévisible, à laquelle il faut répondre en choisissant parmi plusieurs actions possibles. C'est exactement le cas du support de premier niveau, et c'est pourquoi le premier chapitre du six-pack lui est consacré.

RAG et connaissance métier

Un modèle ne connaît pas vos tarifs, vos procédures ni vos délais de livraison. La question n'est donc pas de lui apprendre quelque chose, mais de lui fournir le bon extrait au bon moment, et cette question est de la plomberie plus que de l'intelligence artificielle.

14.1 Ce que le procédé résout

La génération augmentée par la recherche consiste à retrouver, dans un corpus, les passages pertinents pour une question, puis à les donner au modèle avec la question. Le modèle ne mémorise rien : il lit ce qu'on lui tend.

Cette description banale a deux conséquences que l'on oublie systématiquement. La première est que **la qualité du dispositif dépend de la recherche, pas du modèle** : un excellent modèle nourri d'extraits hors sujet répond mal, et changer de modèle n'y remédie pas. La seconde est que **tout ce qui entre dans le corpus devient consultable** par quiconque parle à l'agent, ce qui fait de la constitution du corpus une décision de sécurité autant qu'une décision technique.

14.2 La chaîne

Le dispositif se lit en deux temps qui n'ont pas lieu au même moment : une chaîne d'ingestion, exécutée quand les documents changent, et une chaîne d'interrogation, exécutée à chaque question.

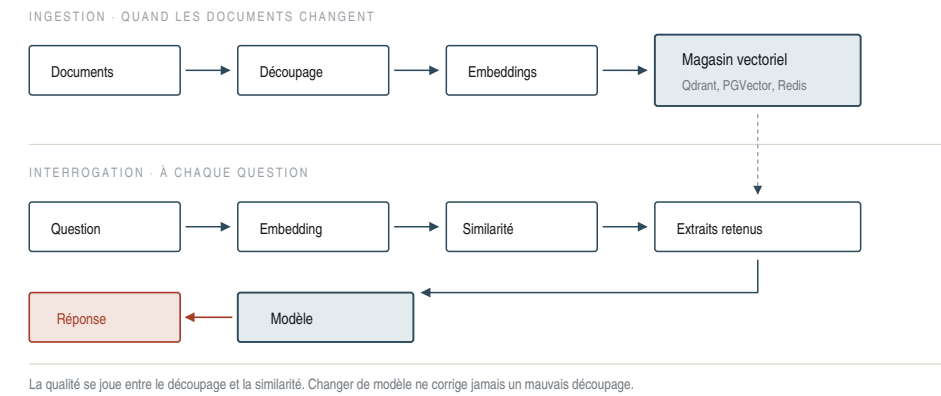


Figure 14.1 Les deux chaînes du procédé. Elles s'exécutent à des moments différents et se conçoivent séparément : la première est un traitement par lots au sens des parties précédentes, la seconde tient dans une conversation.

14.3 Le nœud de magasin vectoriel

Un même nœud sert aux deux chaînes, et son mode décide de tout. Quatre modes existent chez la plupart des fournisseurs, cinq chez ceux qui gèrent la mise à jour.

Insert Documents alimente le magasin : c'est la chaîne d'ingestion. **Get Many** interroge et renvoie les documents les plus proches avec leur score, en nœud ordinaire, ce qui en fait l'outil de mise au point par excellence. **Retrieve Documents comme magasin pour une chaîne** alimente un récupérateur branché sur une chaîne de questions-réponses. **Retrieve Documents comme outil d'agent** branche le magasin directement sur le connecteur d'outils d'un agent, qui décide lui-même quand l'interroger.

Le mode Get Many mérite une insistance particulière. C'est le seul qui montre les scores de similarité, donc le seul qui permette de savoir si la recherche fonctionne. Brancher directement un magasin sur un agent et constater que les réponses sont mauvaises ne dit pas où est le problème ; interroger le magasin en mode ordinaire avec dix questions types et regarder ce qui remonte le dit en dix minutes.

Une option de reclassement est disponible sur les trois modes de lecture. Elle ajoute une seconde passe qui réordonne les résultats et améliore sensiblement la pertinence, au prix d'un appel supplémentaire. Des filtres sur les métadonnées permettent par ailleurs de restreindre la recherche, ce qui est le mécanisme à utiliser pour cloisonner un corpus par client ou par service.

MÉTHODE

Toujours mettre au point la recherche en mode Get Many, avec les scores sous les yeux, avant de la brancher sur un agent.

14.4 Le piège du magasin en mémoire

Le magasin simple, qui vit dans l'instance, est parfait pour apprendre et dangereux en production. Sur l'offre hébergée, des limites de taille et d'expiration sont préréglées ; en auto-hébergement, les deux valeurs sont à l'infini par défaut. Un magasin simple alimenté par un workflow planifié grossit donc sans fin et sans nettoyage, jusqu'à ce que l'instance manque de mémoire.

La règle est simple : le magasin simple sert au prototype, un magasin externe sert au reste. En mode queue, la question ne se pose même pas, puisque les ouvriers ne partagent pas leur mémoire.

14.5 Les tables de données

Toute la connaissance métier n'a pas vocation à devenir vectorielle. Une grille tarifaire, une liste de statuts, une table de correspondance entre codes se cherchent par égalité, pas par similarité, et les mettre dans un magasin vectoriel produit des réponses approximatives là où une correspondance exacte était possible.

Les tables de données répondent à ce besoin, avec un plafond de deux cents mébioctets par installation par défaut, un avertissement à quatre-vingts pour cent et des erreurs d'exécution au-delà. Elles ne sont pas accessibles depuis un nœud Code, ce qui oriente vers les nœuds dédiés. Exposées comme outil, elles donnent à l'agent une réponse exacte là où le corpus documentaire donnerait une réponse plausible.

14.6 Ce qui fait la qualité

Quatre leviers, par ordre d'effet décroissant, et le premier compte davantage que les trois autres réunis.

- **Le découpage.** Des fragments trop courts perdent leur contexte, trop longs noient l'information pertinente. Le découpage doit suivre la structure du document, par section ou par procédure, plutôt qu'un nombre de caractères arbitraire.
- **Les métadonnées.** Attacher à chaque fragment sa source, sa date et son périmètre permet de filtrer, de citer et de retirer un document obsolète sans reconstruire le magasin.
- **Le reclassement.** Une seconde passe sur les résultats corrige une part notable des recherches médiocres.
- **Le nombre d'extraits.** Plus n'est pas mieux : au-delà d'une poignée, on ajoute du bruit et du coût.

Un dernier point relève de la conception et non du réglage : citer la source. Une réponse qui indique de quel document elle provient est vérifiable par l'utilisateur, et cette vérifiabilité vaut mieux qu'un gain de précision. Elle transforme une réponse qu'il faut croire en une réponse qu'on peut contrôler.

Orchestration multi-agents

Multiplier les agents est la réponse la plus séduisante et la plus souvent inutile aux limites d'un agent unique. Ce chapitre donne les deux patrons qui fonctionnent et le critère qui permet de choisir entre eux, ou de n'en choisir aucun.

15.1 Le réflexe à interroger

Quand un agent unique donne des résultats moyens, l'instinct suggère d'en ajouter un second, spécialisé. Dans la majorité des cas, le problème n'était pas là : il venait d'outils mal décrits, d'une recherche documentaire médiocre, ou d'instructions qui demandaient trop de choses à la fois.

La question à se poser avant toute architecture multi-agents est donc brutale : ai-je déjà réduit le nombre d'outils, réécrit leurs descriptions et vérifié ma recherche en mode ordinaire ? Si la réponse est non, un second agent ne fera qu'ajouter du coût et une source d'erreur.

15.2 Deux patrons

Quand la multiplication se justifie vraiment, deux structures s'offrent, et elles ne se valent pas selon les cas.

Le **routage** place devant les agents un classificateur déterministe, ou un nœud de classification de texte, qui décide du domaine puis aiguille vers un sous-workflow spécialisé. Chaque branche est un traitement complet et prévisible ; le non-déterminisme est confiné à la seule décision d'aiguillage, qui est observable et testable. C'est le patron à préférer quand les domaines sont connus et peu nombreux.

La **délégation** place un agent principal qui dispose d'autres agents comme outils. Il décide lui-même à qui confier quoi, et peut enchaîner plusieurs délégations dans un même tour. C'est plus souple et moins prévisible, et cela convient quand la demande ne se range pas dans des catégories fixées d'avance.

CRITÈRE

Le domaine est-il connu d'avance, ou faut-il le déduire de la demande ?

Le nœud d'agent utilisable comme outil rend le second patron direct, avec une imbrication possible sur plusieurs niveaux et sans la gestion de contexte qu'imposerait un sous-workflow. Cette facilité est aussi son danger : rien n'empêche de construire une hiérarchie de quatre niveaux dont plus personne ne peut prédire le comportement ni le coût.

15.3 Le coût réel

Chaque niveau de délégation multiplie les appels. Un agent principal qui délègue à un spécialiste, lequel appelle deux outils, produit au minimum quatre échanges avec un modèle là où un agent unique en aurait produit deux. La latence suit, et sur un canal conversationnel où quelqu'un attend, elle se voit.

Le raisonnement à tenir est celui du chapitre 1, appliqué ici : le gain de qualité doit être mesuré, pas supposé. Les évaluations du chapitre suivant sont l'outil qui permet de dire si le second agent améliore réellement quelque chose, et c'est le seul argument recevable pour le conserver.

15.4 Une règle de conception

Trois limites tirées de l'expérience, qui évitent les architectures dont on ne sort plus.

- **Un seul niveau de délégation** par défaut. Le deuxième niveau doit être justifié par une mesure, pas par une élégance apparente.
- **Les effets externes restent dans des sous-workflows déterministes.** Un agent délégué peut lire, chercher, résumer ; il n'écrit qu'à travers un sous-workflow qui porte l'idempotence et le contrôle.
- **Chaque agent a une liste d'outils courte.** Au-delà d'une poignée, la description des outils devient le facteur limitant, et la spécialisation qu'on cherchait par l'architecture s'obtient plus sûrement en réduisant la liste.

Fiabilité et gouvernance des systèmes IA

Le chapitre 7 traitait des échecs qui se voient. Celui-ci traite des échecs qui ne se voient pas : une réponse fausse mais fluide, une donnée personnelle recopiée dans un journal, un outil appelé sur la foi d'une instruction glissée dans un message entrant.

16.1 Ce qui change par rapport au chapitre 7

Un workflow déterministe échoue bruyamment : un nœud rouge, un message, une trace. Un système à base de modèle échoue silencieusement, en produisant une sortie bien formée et fausse. Aucun des mécanismes de la partie II ne détecte cela, parce qu'il n'y a rien à détecter du point de vue technique.

Trois dispositifs distincts répondent à ce problème, et il faut les trois : des garde-fous qui filtrent ce qui entre et ce qui sort, une approbation humaine sur les actions irréversibles, et des évaluations qui mesurent la qualité dans le temps. Aucun ne remplace les deux autres.

16.2 Les garde-fous

Le nœud dédié fonctionne selon deux modes qu'il ne faut pas confondre.

En mode **contrôle**, il vérifie et dirige les cas non conformes vers une branche d'échec, exactement comme la sortie d'erreur du chapitre 7. En mode **assainissement**, il laisse passer en remplaçant les éléments détectés par des marqueurs : données personnelles, clés secrètes, adresses, motifs définis par expression régulière.

Les contrôles couvrent la tentative de détournement d'instructions, le contenu inapproprié, l'alignement thématique, les données personnelles, les clés secrètes, les adresses et les motifs personnalisés. Les premiers appellent un modèle, les seconds non, ce qui a une conséquence de coût traitée plus loin.

QUATRE POINTS D'INSERTION

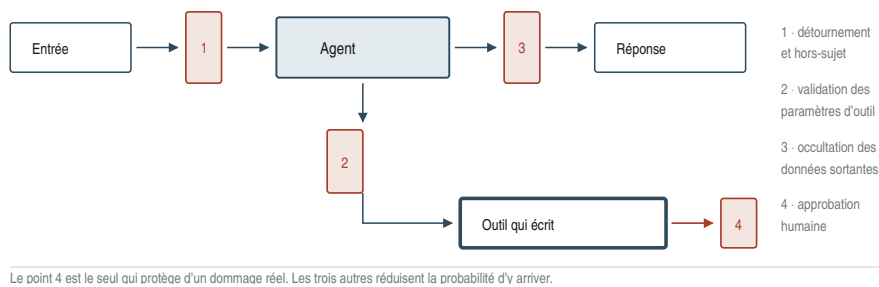


Figure 16.1 Points d'insertion des garde-fous et de l'approbation humaine. La hiérarchie compte : un filtre d'entrée réduit un risque, un point d'approbation avant l'effet externe le supprime.

16.3 L'approbation humaine au niveau de l'outil

Depuis la version 2.6, l'approbation ne se place plus seulement dans le flux : elle se règle outil par outil. Un agent peut disposer librement des outils de lecture et voir chacun de ses appels d'écriture soumis à validation avant exécution.

C'est le mécanisme le plus important de tout ce chapitre, et le raisonnement du chapitre 4 s'y applique directement. Le contrôle se place avant les effets irréversibles ou coûteux, il arrive avec le contexte suffisant pour décider vite, il porte sur une fraction des cas plutôt que sur tous, et il a une échéance avec un comportement par défaut. Un seuil de montant ou un score de confiance réduit souvent le volume d'approbations de façon considérable sans réduire la protection.

Un agent qui n'a que des outils de lecture et un point d'approbation sur les écritures peut être mis en production avec une sérénité qu'aucun réglage d'instructions ne procure.

LE POINT QUI COMPTE

C'est le seul mécanisme qui empêche un dommage plutôt que de le rendre moins probable.

16.4 Les évaluations

Sans mesure, la qualité d'un système à base de modèle est une opinion, et cette opinion change à chaque modification d'instruction sans que personne puisse dire si l'on a progressé ou reculé.

Le mécanisme repose sur un jeu de cas comportant des entrées et les sorties attendues. Le workflow s'exécute sur chaque cas et des métriques comparent le résultat à l'attendu. Plusieurs sont fournies : la justesse et l'utilité, notées de un à cinq par un modèle évaluateur ; la similarité de chaîne, entre zéro et un ; la catégorisation, qui vaut un ou zéro selon la correspondance exacte ; et la vérification des outils appelés. Des métriques personnalisées permettent de mesurer ce qui compte pour votre métier.

Deux détails pratiques. Une opération permet de savoir si l'on se trouve en cours d'évaluation, ce qui autorise à ne calculer les métriques coûteuses que dans ce contexte et non à chaque exécution de production. Et la concurrence

des évaluations suit le niveau de licence, ce qui rend les campagnes lentes sur les offres basses : mieux vaut un jeu de vingt cas bien choisis, rejoué à chaque modification, qu'un jeu de trois cents qu'on ne relance jamais.

LA DISCIPLINE MINIMALE

Un jeu de cas versionné dans le dépôt, à côté du workflow. Il est rejoué avant chaque publication, exactement comme on relit une comparaison de versions au chapitre 11. Une modification d'instruction qui fait baisser une métrique n'est pas publiée, et cette règle simple suffit à empêcher la dérive lente qui caractérise ces systèmes.

16.5 Ce que les garde-fous coûtent

Les contrôles reposant sur un modèle ajoutent un appel, donc une latence et un prix, à chaque passage. Sur un canal conversationnel, cela reste marginal au regard du coût de l'agent lui-même. Sur un traitement par lots de dix mille enregistrements, c'est le mauvais outil : l'occultation des données doit alors se faire en amont, par transformation ou au niveau de la base, avant que le lot n'atteigne le modèle.

La règle est celle du chapitre 6 appliquée ici : réduire le volume est toujours préférable à filtrer le volume.

16.6 Ce qu'on écrit dans le contrat

Cette section s'adresse à la piste décideur, et elle vaut d'être lue avant de signer quoi que ce soit.

Un système à base de modèle ne se livre pas avec une garantie de justesse, parce que personne ne peut l'accorder. Ce qui se livre, et ce qui doit figurer noir sur blanc, est différent : le périmètre des actions que le système peut entreprendre seul, la liste des actions soumises à approbation humaine, le taux de couverture attendu et la procédure pour les cas non couverts, les métriques suivies avec leur niveau de référence, et le délai de correction en cas de dérive constatée.

Un prestataire qui promet un taux de justesse est soit imprudent, soit en train de vendre quelque chose qu'il ne contrôle pas. Un prestataire qui livre un périmètre, des points de contrôle et une mesure vend ce qu'il peut effectivement tenir.

CE QUE LA PARTIE V AJOUTE

Seize chapitres ont décrit des mécanismes. Les six suivants les mettent au travail sur des situations réelles, chacune avec son commanditaire, ses contraintes, ses pièges rencontrés et son coût. L'instance reste la même d'un cas à l'autre, comme annoncé au chapitre 2 ; les métiers, eux, changent à chaque chapitre.

Assistant WhatsApp de premier niveau

PISTE PRATICIEN ET PISTE DÉCIDEUR

Gabarit des six chapitres : la situation, ce qu'on n'automatise pas, la chaîne, les décisions de conception, les pièges rencontrés, le coût, les variantes.

Le cas qui rentabilise le canal, à condition de comprendre que sa rentabilité tient à un détail de facturation et non à la qualité de l'assistant.

17.1 La situation

Une structure de services reçoit désormais plus de messages WhatsApp que d'appels téléphoniques. Trois personnes y répondent, en alternance, sur un téléphone partagé. Une part importante des échanges porte sur les mêmes questions : les horaires, les tarifs, la disponibilité, l'adresse, l'état d'avancement d'un dossier.

Le comptage préalable, mené sur deux semaines conformément au chapitre 1, donne le chiffre qui décide : un peu plus de la moitié des messages entrants appellent une réponse figée, disponible dans un document que personne ne consulte. Le reste demande un jugement, une vérification, ou porte sur un dossier précis.

17.2 Ce qu'on n'automatise pas

La décision de périmètre est prise avant toute conception, et elle est plus restrictive que ce que le client demandait. L'assistant répond aux questions générales. Il ne consulte aucun dossier, ne prend aucun engagement de délai, ne donne aucun prix négocié, et ne discute jamais d'une réclamation.

Cette restriction n'est pas de la prudence excessive : elle correspond exactement à la frontière du chapitre 4 entre ce qui a une règle écrite et ce qui relève du jugement. Elle a aussi un effet secondaire appréciable, celui de rendre le corpus documentaire public par nature, donc sans enjeu de confidentialité au sens du chapitre 14.

PÉRIMÈTRE

L'assistant répond aux questions générales. Tout ce qui touche un dossier nommé passe à un humain.

17.3 La chaîne

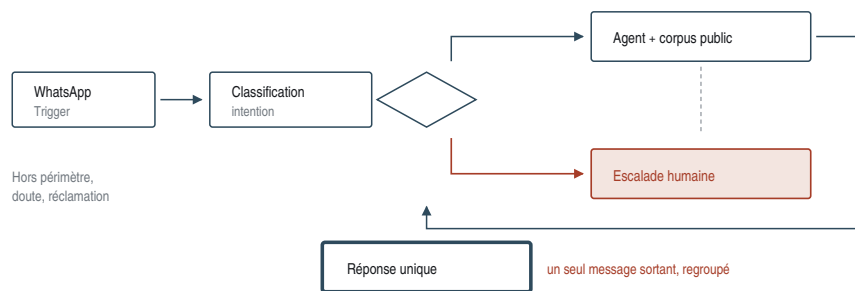


Figure 17.1 La chaîne complète. La classification est déterministe avant d'être confiée à un modèle, et le point de sortie est unique : une seule réponse regroupée, pour la raison de coût exposée au chapitre 12.

17.4 Les décisions de conception

Classifier avant de répondre. Le premier réflexe consiste à brancher un agent sur le message entrant. Le bon geste est d'insérer avant lui une classification d'intention, qui décide si la question relève du périmètre. Ce filtre coûte peu, se teste avec le jeu de cas du chapitre 16, et surtout confine le non-déterminisme à une décision unique et observable.

Regrouper la réponse. Le chapitre 12 l'impose : chaque message sortant se paie à partir du 1^{er} octobre 2026. Un assistant qui répond en trois bulles successives coûte trois fois un assistant qui répond en une. La consigne au modèle en tient compte explicitement.

Soigner l'escalade. C'est le point où la plupart des projets se cassent. Passer la main ne consiste pas à écrire « un conseiller va vous répondre » : il faut transmettre à l'humain le fil complet, l'intention détectée et le motif de l'escalade, dans le canal où il travaille. Une escalade mal faite oblige le client à tout répéter, ce qui produit une expérience pire que l'absence d'assistant.

17.5 Les pièges rencontrés

- **Le webhook unique.** Traité au chapitre 12, il se manifeste ici de la façon la plus déplaisante : le premier test depuis l'éditeur coupe silencieusement la production. Deux applications Meta distinctes règlent le problème, à condition de l'avoir prévu.
- **La vérification du compte.** Une à deux semaines, à lancer le premier jour. C'est le seul élément du projet qui ne s'accélère pas.
- **La fenêtre qui se ferme.** Un client qui écrit à vingt-trois heures et qu'on rappelle le surlendemain se trouve hors fenêtre : la réponse libre devient impossible et il faut un gabarit approuvé. Le workflow doit donc connaître l'âge du dernier message entrant, et basculer de mode en conséquence.

- **Le refus pour fréquence.** Le plafond marketing produit un refus qui n'est pas un incident technique. Le traiter comme tel remplit la boîte d'alertes et masque les vrais problèmes.

17.6 Le coût

Trois lignes, et une seule est significative.

Le **modèle** coûte quelques centimes par conversation traitée, montant négligeable au regard du temps humain qu'il remplace. La **plateforme** ne coûte rien de plus en auto-hébergement, un canal supplémentaire ne changeant pas la facture. Le **canal** est la seule ligne qui compte, et elle change de nature le 1^{er} octobre 2026 : ce qui était gratuit devient facturé au message.

La conséquence est nette et vaut d'être dite au client avant la signature : la rentabilité du cas dépend du nombre de messages sortants par conversation, pas du nombre de conversations. Un assistant bavard peut coûter plus cher que les trois personnes qu'il soulage. C'est la raison pour laquelle le regroupement des réponses est une décision d'architecture et non une préférence de style.

MESURE

Le champ de tarification journalisé donne le coût réel par conversation, en francs, sans estimation.

17.7 Variantes

Le même dispositif se transpose au canal web du chapitre suivant, où il coûte moins cher parce qu'aucun message n'est facturé, et où il touche moins de monde parce qu'il faut visiter le site. Il se transpose aussi au courriel, avec une contrainte inverse : le coût disparaît et le délai de réponse cesse d'être un critère.

Une variante à écarter mérite d'être nommée : brancher l'assistant sur les dossiers clients dès la première version. Elle transforme un cas simple et sans risque en un cas qui touche des données personnelles, exige les garde-fous du chapitre 16 et déplace le point d'approbation. Elle se fait, mais en deuxième temps et avec le budget correspondant.

Agent de support documenté

Une base documentaire que personne ne consulte n'est pas un problème de documentation. C'est un problème d'accès, et il se règle par la recherche plutôt que par la réécriture.

18.1 La situation

Une équipe technique dispose de procédures, de notes d'installation, de comptes rendus d'incidents et d'un manuel produit. L'ensemble représente quelques centaines de pages réparties entre des documents, des pages internes et des fichiers déposés au fil des ans. Chacun sait que la réponse existe quelque part ; personne ne sait où.

Le symptôme est mesurable et c'est lui qu'on mesure avant de commencer : le nombre de questions posées à l'oral qui trouvent leur réponse dans un document existant. Ce chiffre justifie ou enterre le projet.

18.2 Le corpus est une décision de sécurité

Le chapitre 14 l'annonçait : tout ce qui entre dans le magasin devient interrogeable par quiconque parle à l'agent. La constitution du corpus est donc la première décision, et elle se prend avec le client, document par document, avant toute ingestion.

Trois catégories se dégagent. Ce qui est diffusable à tous entre sans discussion. Ce qui est réservé à un service donné entre avec un filtre sur les métadonnées, mécanisme prévu par les nœuds de magasin et seul cloisonnement fiable. Ce qui contient des données personnelles ou des éléments contractuels n'entre pas dans la première version, quelle que soit l'insistance.

RÈGLE SIMPLE

Un document dont on hésite à dire s'il peut être lu par tous les utilisateurs de l'agent n'entre pas dans le corpus de la première version.

18.3 La chaîne

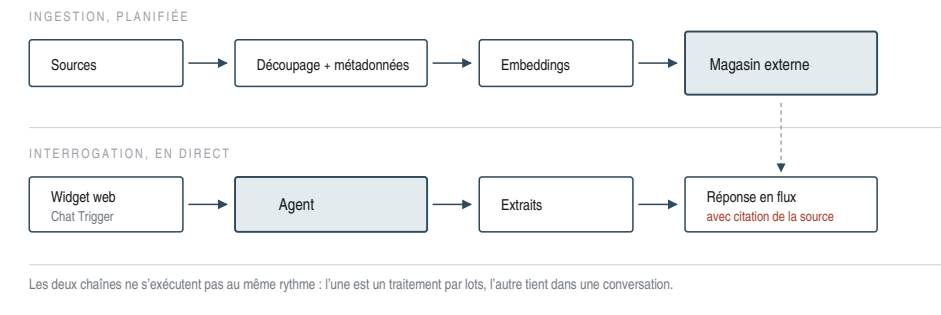


Figure 18.1 Architecture du cas. La citation de la source n'est pas un ornement : elle transforme une réponse qu'il faut croire en une réponse qu'on peut vérifier, ce qui est le seul moyen d'obtenir l'adhésion d'une équipe technique.

18.4 Le widget et ses contraintes

Le composant s'intègre dans une page interne, en mode embarqué. Deux contraintes du chapitre 12 se rencontrent immédiatement.

La première est que le nœud Chat n'est pas supporté dans ce mode : la réponse passe par le nœud de réponse au webhook. La seconde est la diffusion en flux, qui exige un mode de réponse continue et qui n'est pas un raffinement ici : la recherche documentaire ajoute une seconde ou deux avant que le modèle ne commence à écrire, et sans flux l'utilisateur conclut que l'outil ne marche pas.

18.5 Le jeu de test

C'est ce qui distingue ce cas d'une démonstration, et c'est ce qui se livre au client en même temps que le workflow.

Une vingtaine de questions réelles, prélevées dans les échanges de l'équipe avant le projet, avec la réponse attendue et le document qui la contient. Le jeu est versionné dans le dépôt à côté du workflow, conformément au chapitre 16, et rejoué avant chaque publication. Les métriques retenues sont la justesse de la réponse et la présence du bon document parmi les extraits retenus, cette seconde mesure étant la plus informative : elle sépare une erreur de recherche d'une erreur de rédaction, et ces deux pannes n'ont pas le même remède.

18.6 Le coût

L'ingestion est un coût unique, proportionnel au volume du corpus, puis un coût marginal à chaque mise à jour. L'interrogation coûte un appel d'embedding et un appel de modèle par question, l'un négligeable et l'autre modeste. Le magasin vectoriel externe est la seule ligne d'infrastructure ajoutée.

Le calcul de retour se fait sur le temps de recherche évité, mesuré avant, et non sur un nombre de questions posées. C'est la nuance du chapitre 1 : le volume d'usage prouve l'adoption, pas la valeur.

18.7 Variantes

Le corpus se cloisonne par filtre sur les métadonnées quand plusieurs services partagent l'agent. La même chaîne alimente l'assistant WhatsApp du chapitre précédent, à condition que le corpus reste public. Et la table de données du chapitre 14 vient compléter le dispositif partout où une réponse exacte vaut mieux qu'une réponse plausible : un tarif, un délai contractuel, un code de statut.

Réconciliation d'encaissements mobile money

PISTE PRATICIEN ET PISTE DÉCIDEUR

Le cas le moins spectaculaire du livre et le plus rentable. Il ne contient aucun modèle de langage, aucune conversation, et il justifie à lui seul le budget d'une mission.

19.1 La situation

Les paiements arrivent par un agrégateur qui regroupe plusieurs opérateurs de monnaie électronique. Quelqu'un consulte le tableau de bord du prestataire à intervalle régulier, reporte les transactions dans un tableur, rapproche chaque encaissement d'une commande, et confirme au client.

La mesure préalable donne deux chiffres. Cinq à dix minutes par transaction, temps de recherche compris quand la référence est ambiguë. Et un taux d'erreur de saisie qui ne se découvre qu'à la clôture, sous forme d'écarts que personne ne sait expliquer trois semaines après.

Le second chiffre est celui qui décide, conformément au chapitre 1 : le coût de l'erreur dépasse largement le coût du temps.

19.2 La clé naturelle

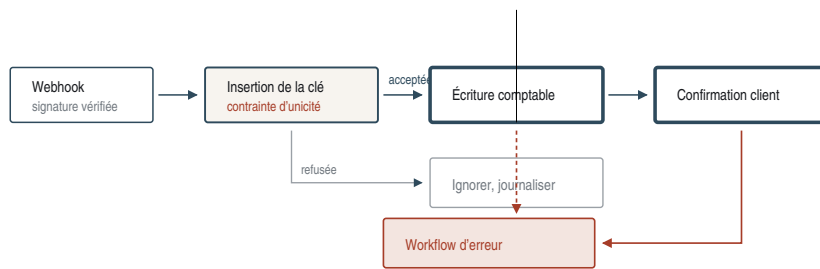
Toute la conception tient dans cette question, posée au client avant d'ouvrir l'éditeur : quelle valeur identifie de manière unique et définitive un encaissement ?

La réponse est la référence de transaction attribuée par le prestataire. Elle est stable, unique, présente dans la notification comme dans le tableau de bord, et surtout elle vient de l'émetteur. Une clé construite localement, à partir d'un horodatage de réception ou d'un identifiant tiré au hasard, changerait à chaque livraison répétée et laisserait passer tous les doublons.

NON NÉGOCIABLE

La clé vient du prestataire. Une clé fabriquée localement change à chaque jeu et ne protège de rien.

19.3 La chaîne



Le contrôle d'unicité est porté par la base et non par un nœud : deux livraisons simultanées ne peuvent pas passer toutes les deux.

Figure 19.1 La chaîne d'encaissement. Les deux effets externes portent un double trait, conformément à la convention du chapitre 4, et l'un comme l'autre se trouvent derrière le contrôle d'unicité.

19.4 Pourquoi une contrainte en base

Le chapitre 7 présentait deux moyens de dédoublonner. Le nœud dédié, qui écarte les items déjà vus lors des exécutions précédentes, convient parfaitement à un flux de veille. Il ne convient pas ici, pour deux raisons qu'il faut savoir énoncer devant un client.

La première est la taille de l'historique, plafonnée et gérée en file : au-delà de la limite, les entrées les plus anciennes disparaissent, et un encaissement ancien qui reviendrait serait traité comme neuf. Sur des mouvements d'argent, ce risque n'est pas acceptable.

La seconde est la simultanéité. Une vérification préalable suivie d'une écriture n'est pas atomique : deux livraisons arrivant en même temps peuvent toutes deux constater l'absence de la clé, puis toutes deux écrire. Une contrainte d'unicité portée par la base refuse la seconde insertion quelle que soit la concurrence. C'est le stockage qui garantit, pas le workflow.

19.5 Les pièges rencontrés

- **Les états intermédiaires.** Une transaction passe par plusieurs statuts avant d'être définitive. Écrire en comptabilité sur un statut non final produit des lignes à corriger. Le workflow ne traite que les états terminaux et journalise les autres.
- **Le rejeu du prestataire.** Certains agrégateurs renvoient la notification tant qu'ils n'ont pas reçu une réponse. Répondre immédiatement, avant tout traitement, est ici le bon mode de réponse au sens du chapitre 6.
- **Le réseau intermittent.** Une coupure pendant l'écriture laisse un état indéterminé. C'est précisément le cas que l'idempotence rend inoffensif : on rejoue, et le second passage est refusé par la contrainte.
- **La référence de commande absente.** Une part des paiements arrive sans rapprochement possible. Ces cas ne sont pas des erreurs : ils constituent le

taux de non-couverture, ils partent vers une file de traitement manuel, et ils sont comptés.

19.6 Le coût et la preuve

Le coût d'exploitation est proche de zéro : quelques exécutions par jour, aucun modèle de langage, aucune infrastructure ajoutée hormis la table de référence.

La preuve, elle, se construit sur trois chiffres relevés avant et après : le temps de traitement par transaction, le nombre d'écarts constatés à la clôture mensuelle, et le délai entre l'encaissement et sa confirmation au client. Le troisième est celui qui intéresse le dirigeant, parce qu'il touche à la relation commerciale et non à la charge administrative.

19.7 Variantes

Le même patron couvre tout encaissement notifié par un tiers : passerelle bancaire, plateforme de commerce, prélèvement. Seule change la forme de la notification ; la clé, la contrainte et la structure restent identiques.

Une extension fréquente consiste à ajouter la relance des paiements en attente. Elle change la nature du cas : on passe d'un traitement réactif à un traitement sortant, avec les coûts de canal du chapitre 12 et les plafonds de fréquence qui vont avec. C'est un autre projet, à chiffrer comme tel.

Devis vers facture depuis canaux mixtes

PISTE PRATICIEN ET PISTE DÉCIDEUR

Trois canaux d'entrée, un seul outil de gestion, et entre les deux plusieurs personnes qui ressaisissent. Le cas classique, et celui où l'on est le plus tenté de mettre un agent là où il ne faut pas.

20.1 La situation

Les demandes arrivent par WhatsApp, par téléphone et par courriel. Elles sont notées sur un carnet, dans un fil de discussion ou dans une boîte de réception, puis ressaisies dans l'outil de gestion par une personne qui n'a pas reçu la demande initiale. Chaque transcription est une occasion de perdre une information et d'en inventer une.

La mesure préalable porte sur deux choses : le délai entre la demande et l'envoi du devis, et la proportion de documents qui doivent être corrigés après émission. Le premier chiffre est celui du chapitre 1 qui déplace le chiffre d'affaires ; le second est celui qui rend le projet urgent aux yeux de la comptabilité.

20.2 Unifier avant de traiter

Le premier geste n'est pas technique. Il consiste à ramener les trois canaux à une forme unique, ce qui est exactement la sortie « standardiser » de la figure 1.1. Les demandes téléphoniques passent par un formulaire court que la personne remplit pendant l'appel. Les courriels et les messages entrent tels quels. À partir de là, un seul format circule.

Cette étape produit une partie du gain à elle seule, sans automatisation, et elle rend le reste possible. Un projet qui la saute construit trois chaînes parallèles qu'il faudra maintenir séparément.

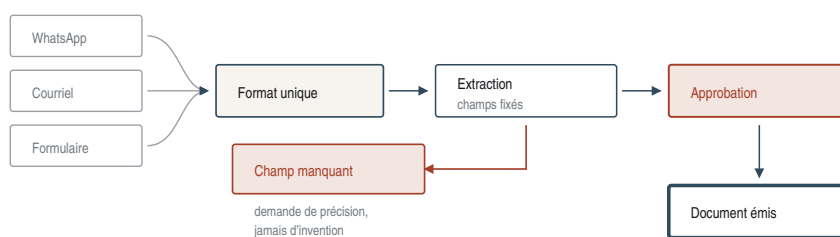
ORDRE DES CHANTIERS

Standardiser la saisie d'abord. Une bonne part du gain arrive avant qu'un seul nœud soit branché.

20.3 Extraction structurée, pas agent

La tentation est de confier la lecture des messages à un agent. C'est le mauvais choix, pour la raison exposée au chapitre 13 : la tâche n'est pas de décider quoi faire, elle est d'extraire des champs connus d'avance depuis un texte libre. Un nœud d'extraction structurée fait cela de façon prévisible, testable, et sans boucle dont le nombre de tours varie.

Les champs sont fixés avec le client : client, articles, quantités, délai souhaité, adresse de livraison. Ce qui n'est pas extractible avec certitude est laissé vide plutôt que deviné, et un champ vide déclenche une demande de précision plutôt qu'une invention plausible. C'est la différence entre un outil qu'on peut relire et un outil qu'il faut vérifier entièrement.



Aucun document engageant ne part sans passer par le point d'approbation, quel que soit le score de confiance de l'extraction.

Figure 20.1 Unification des canaux puis extraction. Le point d'approbation est placé avant le seul effet externe irréversible du cas, selon la règle du chapitre 4 reprise au chapitre 16.

20.4 L'approbation avant émission

Un devis est un document engageant. Il part avec un point d'approbation, sans exception, et ce point respecte les trois conditions du chapitre 4 : il arrive avec tout le contexte, il a une échéance, et il porte sur une fraction des cas.

La fraction se règle par un seuil. En dessous d'un montant convenu et lorsque tous les champs ont été extraits sans ambiguïté, le document part directement. Au-dessus, ou dès qu'un champ a demandé une précision, il passe en validation. Ce réglage réduit fortement le volume de contrôle sans réduire la protection, et il se durcit ou s'assouplit au fil des mois selon ce que montrent les mesures.

20.5 Les pièges rencontrés

- **Le message en trois parties.** Sur WhatsApp, un client écrit rarement tout d'un coup. Trois messages successifs produisent trois déclenchements et trois extractions incomplètes. La parade est une fenêtre d'attente courte avant traitement, qui regroupe les messages d'une même session.
- **Le même client sur deux canaux.** Une demande envoyée par courriel puis répétée par WhatsApp crée deux documents. La déduplication ne peut pas reposer sur une clé d'émetteur ici, puisqu'il n'y en a pas de commune : elle

repose sur un rapprochement par client et par fenêtre de temps, avec passage en validation en cas de doute.

- **Les pièces jointes illisibles.** Une photo de bon de commande prise de travers fait partie du taux de non-couverture. Elle part en traitement manuel, et ce volume est compté.
- **L'écriture dans l'outil de gestion.** C'est un effet externe, donc il a sa clé d'idempotence et son sous-workflow dédié, comme au chapitre 19.

20.6 Le coût

L'extraction coûte un appel de modèle par demande, montant sans commune mesure avec le temps de ressaisie évité. La ligne réelle est ailleurs : dans le temps d'approbation, qui reste humain et qu'il faut compter dans le calcul du chapitre 1 plutôt que de l'oublier parce qu'il est court.

Le retour se lit sur le délai de production du devis, et c'est le chiffre à mettre en avant. Un devis envoyé dans l'heure au lieu du surlendemain ne fait pas économiser deux jours de salaire ; il fait gagner des affaires.

20.7 Variantes

La chaîne se prolonge naturellement vers la facturation, avec le même patron et un point d'approbation qui peut disparaître dès lors que la facture découle mécaniquement d'un devis déjà validé. Elle se prolonge aussi vers la relance, qui relève du canal sortant et de son économie.

Une variante à écarter : confier au modèle le calcul des prix. Le prix vient de la grille tarifaire, qui est une table de données au sens du chapitre 14, interrogée par correspondance exacte. Un prix approximatif est un prix faux.

Agent d'opérations interne exposé en MCP

PISTE PRATICIEN ET PISTE DÉCIDEUR

Le cas qui renverse la relation habituelle : au lieu que n8n consomme des services, n8n expose ses workflows comme outils à un assistant extérieur.

21.1 La situation

L'équipe technique veut interroger et agir sur les systèmes internes depuis l'assistant qu'elle utilise déjà, sans ouvrir d'accès direct aux bases ni distribuer des identifiants de production. Les questions sont ordinaires : où en est telle commande, quel est l'état du service, combien d'exécutions ont échoué cette nuit, relance le traitement de tel dossier.

La réponse habituelle consiste à écrire des scripts et à les distribuer. Elle échoue toujours de la même façon : les scripts divergent, les identifiants se recopient, et personne ne sait plus qui a exécuté quoi.

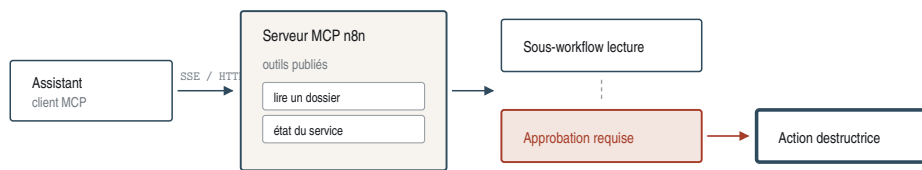
21.2 Exposer des workflows comme outils

Le déclencheur de serveur MCP transforme un ensemble de workflows en outils appelables par un client extérieur. Chaque outil est un workflow ordinaire, avec ses identifiants, ses contrôles, son idempotence et son workflow d'erreur. L'assistant ne voit qu'un nom, une description et des paramètres.

C'est l'inversion qui donne au cas sa valeur. Les accès de production ne quittent pas l'instance. La surface exposée est exactement la liste des workflows publiés comme outils, ni plus ni moins, et l'ajout d'une capacité est un acte de publication au sens du chapitre 11, donc soumis aux droits du chapitre 9.

LE POINT FORT

Les identifiants restent dans n8n. L'assistant n'a jamais accès aux systèmes, seulement aux opérations qu'on a décidé d'exposer.



Lecture libre, écriture sous approbation. La frontière ne se règle pas dans l'instruction de l'agent : elle se règle outil par outil.

Figure 21.1 n8n en serveur d'outils. La séparation entre lecture et écriture n'est pas une consigne donnée au modèle mais une propriété de la configuration, ce qui la rend indépendante de la formulation des demandes.

21.3 Les contraintes de transport

Deux limites techniques doivent être connues avant de promettre quoi que ce soit.

Le transport se fait par flux d'événements ou par HTTP continu ; l'entrée-sortie standard n'est pas supportée. Certains clients ne parlent que ce dernier mode et exigent alors un pont, ce qui ajoute une pièce à maintenir.

La seconde limite prolonge le chapitre 8. Ces connexions sont durables, et en présence de plusieurs répliques de traitement des webhooks, une connexion établie sur l'une puis reprise sur une autre se coupe. Le trafic MCP doit donc être routé vers une réplique dédiée, ou bénéficier d'une persistance de session au niveau du répartiteur. C'est exactement le problème de la persistance de session du multi-main, et il se règle de la même façon.

21.4 L'approbation par outil

La règle du chapitre 16 s'applique ici sous sa forme la plus stricte, parce que le demandeur n'est plus un client mais un membre de l'équipe, ce qui abaisse naturellement la vigilance.

Les outils de lecture sont libres. Toute opération qui écrit, relance, annule ou supprime passe par une approbation explicite, avec le détail de ce qui va être fait. Cette approbation ne se règle pas dans les instructions de l'agent : elle se règle sur l'outil lui-même, ce qui la rend insensible à la formulation de la demande.

21.5 La trace

C'est le second bénéfice du cas, et il est souvent celui qui emporte la décision. Chaque appel produit une exécution n8n, donc une entrée dans l'historique, avec l'auteur, les paramètres, le résultat et l'horodatage. La journalisation d'audit du chapitre 10 complète le tableau côté configuration.

Là où des scripts distribués ne laissaient rien, on obtient un registre complet de qui a demandé quoi, ce qui a été approuvé et ce qui a été exécuté. Pour

beaucoup d'organisations, cette traçabilité vaut davantage que le gain de temps.

21.6 Variantes

La relation s'inverse tout aussi bien : n8n peut être client d'un serveur MCP extérieur, ce qui donne à ses agents des outils qu'il n'a pas à écrire. Les deux sens coexistent sur la même instance.

Une variante à traiter avec prudence consiste à exposer ces outils au-delà de l'équipe technique. La surface devient alors publique au sens du chapitre 9, et l'ensemble des garde-fous du chapitre 16 redevient nécessaire, en particulier le contrôle des tentatives de détournement d'instructions.

Veille, reporting et preuve du ROI

PISTE PRATICIEN ET PISTE DÉCIDEUR

Le sixième cas ne produit pas de valeur : il la démontre. C'est celui qui finance les cinq autres, et c'est aussi le seul que personne ne demande.

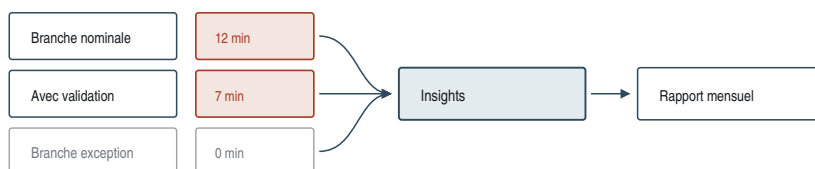
22.1 La situation

Cinq automatisations tournent. Elles fonctionnent, personne ne s'en plaint, et personne ne sait ce qu'elles rapportent. À la discussion budgétaire suivante, la ligne est examinée par quelqu'un qui n'a jamais vu les workflows et qui ne dispose d'aucun chiffre. Ce qui se passe alors ne dépend pas de la qualité du travail accompli.

Ce chapitre construit le document qui manque, et il le construit depuis l'outil plutôt que depuis un tableur, pour la raison énoncée au chapitre 10 : un chiffre produit par la plateforme est vérifiable, un chiffre saisi à la main est une affirmation.

22.2 La chaîne

SUR CHAQUE WORKFLOW DE PRODUCTION



Une valeur par branche, pas une valeur par workflow. C'est ce qui rend le chiffre défendable devant quelqu'un qui cherche à le contester.

Figure 22.1 Instrumentation du temps gagné. Un dossier passé par la branche d'exception n'économise rien et doit compter pour zéro : c'est cette honnêteté de mesure qui rend l'ensemble crédible.

22.3 Une valeur par branche

Le nœud de temps gagné se place sur chaque branche, avec une valeur différente. Un dossier traité entièrement automatiquement économise le temps

complet du traitement manuel. Un dossier passé par une validation humaine économise ce temps moins celui de la validation. Un dossier tombé en exception n'économise rien.

Cette granularité est ce qui distingue une mesure d'une estimation. Un chiffre global multiplié par le volume s'effondre à la première objection, parce que tout le monde sait que tous les dossiers ne se ressemblent pas. Un chiffre agrégé branche par branche résiste, parce qu'il incorpore déjà l'objection.

22.4 Le piège de la rétention

Le chapitre 10 le signalait et il se manifeste précisément ici. Le rapport annuel demande une fenêtre d'un an ; si la rétention des indicateurs est plus courte, le graphique est vide et l'on conclut à une panne. La rétention des indicateurs se règle indépendamment de celle des exécutions, et c'est elle qu'il faut porter à la hauteur de la fenêtre de reporting, dès la mise en production et non le jour du bilan.

La rétention des exécutions, elle, répond à un autre besoin : la capacité d'investiguer un incident. Les deux durées n'ont pas de raison d'être identiques, et les confondre coûte soit de l'espace disque, soit un rapport vide.

FAUX INCIDENT

Un rapport annuel vide quand la rétention couvre trente jours. Vérifier le réglage avant de chercher une panne.

22.5 Le rapport

Quatre chiffres, une demi-page, produits automatiquement et envoyés le premier lundi du mois.

- **Le volume traité**, par workflow, qui montre l'usage réel plutôt que l'usage supposé.
- **Le taux de couverture**, c'est-à-dire la proportion de cas passés sans intervention. C'est le chiffre du chapitre 1, mesuré au lieu d'être promis.
- **Le temps gagné**, agrégé depuis les branches.
- **Les incidents**, avec leur cause et leur délai de résolution.

Le quatrième point est celui qu'on est tenté de retirer et qu'il faut garder. Un prestataire qui publie ses propres incidents avant qu'on les découvre obtient une confiance que ses concurrents n'obtiennent pas, et cette confiance vaut plus qu'un mois sans panne.

22.6 Ce que ce cas finance

Le calcul du chapitre 1 se ferme ici. Le gain net y était une formule avec des termes estimés ; il devient une soustraction entre des mesures. La ligne de maintenance, provisionnée à la signature, se révisé avec les chiffres réels au bout de six mois, comme annoncé.

C'est aussi ce chapitre qui rend possible le refus argumenté. Une automatisation dont le temps gagné mesuré ne couvre pas sa maintenance doit être arrêté.

tée, et le dire soi-même, chiffres à l'appui, est ce qui distingue un partenaire d'un fournisseur.

CE QUE LA PARTIE VI AJOUTE

Les six cas sont livrés et mesurés. Reste ce qui arrive ensuite : la version 3.0, qui retire des nœuds sur lesquels une partie de ce parc repose, et la liste de contrôle qui accompagne toute mise en production. C'est l'objet du dernier chapitre.

Migrer vers n8n 3.0 et checklist de production

Un parc d'automatisations non maintenu se dégrade en deux versions majeures. Ce chapitre traite la prochaine rupture et consolide, en une liste unique, tout ce que les vingt-deux chapitres précédents exigent avant une mise en production.

23.1 Pourquoi ce chapitre existe

La plateforme évolue par nettoyages successifs. La version 2.0 a retiré des comportements sur lesquels des installations reposaient ; la 3.0 retire des nœuds entiers. Ces suppressions sont documentées et pourvues d'un chemin de remplacement, mais elles ne s'appliquent pas toutes seules : un workflow qui utilise un nœud retiré cesse simplement de fonctionner après la mise à jour.

Le risque réel n'est donc pas la difficulté technique de chaque remplacement, qui est faible. C'est de ne pas savoir combien de workflows sont concernés, ni lesquels. Une installation de trente workflows dont personne ne tient l'inventaire découvre le problème le lendemain de la mise à jour, sur un cas de production.

23.2 Ce que la version 3.0 retire

Le tableau ci-dessous rassemble les changements documentés à la date de rédaction. La page officielle des ruptures est complétée à l'approche de la sortie : elle doit être relue le jour de la migration.

CE QUI DISPARAÎT	CE QUI LE REMPLACE
Installation par npm	Déploiement Docker. Pour une installation locale, Docker Compose est le chemin le plus simple.
Nœud Function	Nœud Code, en mode exécution unique pour tous les items.
Nœud Function Item	Nœud Code, en mode exécution pour chaque item.
Nœud Item Lists	

CE QUI DISPARAÎT	CE QUI LE REMPLACE
	Selon l'opération utilisée : Split Out, Aggregate, Sort, Limit, Remove Duplicates ou Summarize.
Nœud Execute Workflow, ancien comportement	Comportement courant, déjà en vigueur depuis la version 2.0.
Version 1 du nœud AI Agent	Version courante. Les workflows déjà en mode Tools Agent se comportent de la même façon après mise à jour.
Mode SQL Agent	Sous-nœud outil Postgres ou MySQL associé à un nœud AI Agent récent.
<code>\$getPairedItem</code>	Chaînage standard : propriété <code>pairedItem</code> OU <code>\$('nom du nœud').item</code> .
Limites de décompression	Plafond par défaut abaissé de 2 Gio à 256 Mio, et nombre d'entrées d'archive de 5 000 à 1 000.

S'ajoutent des durcissements de sécurité par défaut, dans la continuité de la version 2.0 : traitement plus strict des noms de ressources sensibles, comportement plus sûr des identifiants, et rotation de clé activée par défaut. Ces changements ne cassent pas un workflow mais peuvent modifier le comportement d'une installation, et méritent d'être lus avant de programmer la fenêtre de migration.

NOUVEAUTÉ À SURVEILLER

Les valeurs par défaut de sécurité se durcissent, avec notamment une rotation de clé activée d'origine.

23.3 Inventorier avant de migrer

L'inventaire est la seule étape qui ne peut pas être sautée, et c'est celle pour laquelle l'API publique du chapitre 11 prend tout son sens. La procédure tient en quatre passes, à mener sur une copie des définitions de workflows plutôt que dans l'interface.

Première passe : les nœuds retirés. Rechercher, dans les définitions, les types correspondant à Function, Function Item et Item Lists. Le résultat donne la liste exacte des workflows à reprendre, avec l'opération employée dans le cas d'Item Lists, qui détermine le nœud de remplacement.

Deuxième passe : les agents. Repérer les nœuds AI Agent en version 1 et relever leur mode. Les agents déjà en Tools Agent se mettent à jour sans changement de comportement ; les autres se reconstruisent, et le cas SQL demande une conception différente puisqu'il passe d'un mode intégré à un outil branché.

Troisième passe : les expressions. Rechercher l'assistant retiré dans l'ensemble des paramètres. C'est la passe la plus ingrate car l'appel peut se trouver dans n'importe quel champ, et la plus utile car son remplacement suit la logique du chapitre 5 : transporter les valeurs plutôt que les remonter à distance.

Quatrième passe : l'hébergement. Vérifier le mode d'installation. Une instance lancée par npm doit être portée sur Docker avant la mise à jour, et cette bascule est un chantier d'infrastructure indépendant du contenu des workflows.

LE CONSEIL QUI FAIT GAGNER LE PLUS DE TEMPS

Épingler la version exacte de l'instance plutôt que de suivre une étiquette mouvante. Une installation qui suit automatiquement la dernière version se retrouvera en 3.0 sans l'avoir décidé, un matin, avec des workflows qui ne démarrent plus. La mise à jour est un acte de production au sens du chapitre 11 : elle se planifie, elle s'annonce, et elle ne se fait pas un vendredi.

23.4 Répéter avant de faire

La migration se répète intégralement sur l'environnement de recette du chapitre 11, avec les données de production les plus proches possible. Trois vérifications comptent davantage que les autres.

- **Les workflows repris s'exécutent sur les cas réels**, pas sur un item de test. Le chapitre 5 a montré que la cardinalité change tout : un remplacement correct sur un item peut être faux sur douze.
- **Les jeux d'évaluation du chapitre 16 sont rejoués** sur les agents reconstruits, et comparés aux mesures d'avant. C'est le seul moyen de savoir si le remplacement a dégradé la qualité, puisqu'un agent reconstruit ne produit pas de message d'erreur quand il répond moins bien.
- **Les identifiants se déchiffrent** sur tous les processus après bascule. Le chapitre 8 a expliqué pourquoi : une clé de chiffrement absente ou différente sur un ouvrier produit des erreurs d'authentification sans rapport apparent avec la cause.

23.5 Après la migration

Deux gestes closent l'opération et sont systématiquement oubliés.

Le premier est de **vérifier les webhooks entrants**. Un redémarrage d'instance ne réenregistre pas toujours les abonnements côté émetteur, et le chapitre 12 a rappelé que certains services n'acceptent qu'une seule adresse. Un webhook silencieux ne produit aucune alerte : il ne se passe simplement rien.

Le second est de **relire les indicateurs** une semaine après. Une migration réussie techniquement peut avoir dégradé un taux de couverture ou allongé une durée moyenne. Les tableaux de bord du chapitre 10 le montrent ; l'absence d'erreur ne le montre pas.

23.6 La liste de contrôle

Voici, rassemblés, les points que ce livre exige avant toute mise en production. Ils renvoient au chapitre qui les traite.

- ☐ Le processus est décrit par écrit et validé par ceux qui l'exécutent.

CH. 1

☐	Les cas non couverts sont nommés, chiffrés, et une procédure manuelle est prévue pour eux.	CH. 1
☐	Chaque effet externe est identifié sur le schéma et porte une clé d'idempotence venue de l'émetteur.	CH. 4, 7
☐	Le compte d'items est vérifié juste avant chaque nœud qui écrit ou envoie.	CH. 3, 5
☐	Les webhooks entrants sont vérifiés, par le nœud dédié ou par un contrôle de signature explicite.	CH. 6
☐	Les nouvelles tentatives sont activées sur les échecs transitoires uniquement.	CH. 7
☐	Les cas métier non traitables s'arrêtent avec un message lisible par un humain.	CH. 7
☐	Un workflow d'erreur est désigné dans les réglages, et il ne contient pas de logique de rattrapage.	CH. 7
☐	La clé de chiffrement est explicite, identique sur tous les processus, sauvegardée ailleurs que la base.	CH. 8, 9
☐	Le stockage binaire est adapté au mode d'exécution, et la purge est configurée sur le mode actif.	CH. 8
☐	Aucun secret ne figure en clair dans un paramètre de nœud.	CH. 9
☐	Les rôles séparent l'édition, la publication et l'exécution.	CH. 9
☐	Quelqu'un de nommé reçoit l'alerte, sur un canal qu'il consulte réellement.	CH. 10
☐	La rétention des indicateurs couvre la fenêtre du rapport attendu.	CH. 10, 22
☐	La version de l'instance est épinglée, et le workflow est publié et non seulement enregistré.	CH. 11, 23
☐	Les agents disposent d'outils de lecture libres et d'une approbation sur toute écriture.	CH. 16
☐	Un jeu d'évaluation versionné existe et est rejoué avant chaque publication.	CH. 16
☐	Un nœud de temps gagné est posé sur chaque branche, avec une valeur propre.	CH. 22

23.7 Le dernier mot

Vingt-trois chapitres pour un outil qui se prend en main en une après-midi : la disproportion est apparente et elle est le sujet du livre. Assembler des nœuds est facile, et c'est pour cette raison que le métier ne se joue pas là.

Il se joue dans la page écrite avant d'ouvrir l'éditeur, dans la clé qui rend un traitement rejouable, dans le message d'erreur qu'un humain peut lire, dans la valeur de temps gagnée posée branche par branche, et dans la capacité à dire non à une automatisation qui ne rembourserait pas sa propre maintenance. Ce sont des gestes modestes et ils constituent l'essentiel de ce qui distingue une démonstration d'un système dont quelqu'un dépend.

ANNEXE A

Conventions de l'instance

RÉFÉRENCE

Ces conventions sont celles de l'instance qui traverse le livre. Elles sont rassemblées ici pour qu'aucun chapitre n'ait à les rappeler, et pour qu'elles se reprennent telles quelles sur une installation neuve.

Nommage des workflows

Un workflow porte le nom du résultat métier qu'il produit, à l'infinitif, précédé de son domaine. La partie qui suit est le déclencheur quand il est utile de le distinguer.

```
Ventes / Traiter une commande entrante  
Ventes / Émettre un devis  
Finance / Réconcilier un encaissement  
Support / Répondre sur WhatsApp  
Interne / Publier le rapport mensuel
```

Les sous-workflows appelés portent un préfixe qui les identifie comme tels, parce qu'ils n'ont pas de déclencheur propre et n'apparaissent jamais seuls dans une liste d'exécutions.

```
[sw] Écrire en gestion  
[sw] Normaliser un client  
[sw] Notifier le client
```

Nommage des nœuds

Un nœud porte ce qu'il fait, à l'infinitif, dans le vocabulaire du métier et non celui de l'outil. La règle a une conséquence directe à la lecture des expressions : `$('Récupérer le client')` se comprend sans ouvrir le nœud.

Organisation en projets

Un projet par client ou par domaine métier. Les identifiants et les tables de données y sont rattachés, ce qui rend l'isolation effective et non déclarative.

Les coffres externes sensibles sont liés au projet plutôt que laissés globaux, conformément au chapitre 9.

Le workflow d'erreur commun

Un seul workflow d'erreur pour l'installation, désigné dans les réglages de chaque workflow de production. Il ne fait que router : identifier la gravité, notifier le canal concerné, enregistrer une trace avec le lien vers l'exécution. Il ne contient aucune logique de rattrapage.

Champs de transport

Un nœud de champs placé juste après chaque déclencheur recopie dans des noms stables les valeurs dont la suite aura besoin : référence de l'événement, identifiant du client, horodatage, canal d'origine. Cette convention rend inutiles la plupart des expressions de remontée et survit aux fusions, pour la raison exposée au chapitre 5.

ANNEXE B

Variables d'environnement de référence

RÉFÉRENCE

Les variables qui reviennent dans le livre, regroupées par sujet. Cette liste n'est pas exhaustive : elle rassemble celles dont l'oubli produit un incident, avec le chapitre qui les traite.

Chiffrement et exécution

VARIABLE	RÔLE	CH.
N8N_ENCRYPTION_KEY	Chiffre les identifiants. Identique sur tous les processus, sauvegardée hors base.	8, 9
EXECUTIONS_MODE	Bascule en mode queue.	8
EXECUTIONS_TIMEOUT	Délai au-delà duquel une exécution est arrêtée.	8
N8N_CONCURRENCY_PRODUCTION_LIMIT	Plafond d'exécutions de production simultanées. Ne s'applique ni aux exécutions manuelles, ni aux sous-workflows, ni aux workflows d'erreur.	8

File et ouvriers

VARIABLE	RÔLE	CH.
QUEUE_BULL_REDIS_HOST	Hôte du courtier de messages.	8
QUEUE_BULL_REDIS_PORT	Port du courtier.	8
N8N_MULTI_MAIN_SETUP_ENABLED	Active la configuration à plusieurs instances principales. Exige une persistance de session au répartiteur.	8

Données binaires et réponses

VARIABLE	RÔLE	CH.
N8N_DEFAULT_BINARY_DATA_MODE		8

VARIABLE	RÔLE	CH.
	Stockage des fichiers. Le mode système de fichiers n'est pas supporté en mode queue.	
<code>N8N_WEBHOOK_RESPONSE_RELAY_SIZE_MAX</code>	Taille maximale, en mébioctets, du message de file transportant une réponse de webhook. Vaut 64 par défaut.	8
<code>N8N_WEBHOOK_RESPONSE_RELAY_OFFLOAD_ENABLED</code>	Décharge le corps vers le stockage binaire au lieu de faire échouer le nœud.	8

Sécurité et observabilité

VARIABLE	RÔLE	CH.
<code>N8N_BLOCK_ENV_ACCESS_IN_NODE</code>	Bloque l'accès aux variables d'environnement depuis le nœud Code. Actif par défaut depuis la 2.0.	9
<code>N8N_OTEL_ENABLED</code>	Active l'émission de traces.	10
<code>N8N_INSIGHTS_MAX_AGE_DAYS</code>	Rétention des indicateurs. À porter à la hauteur de la fenêtre de reporting attendue.	10, 22

Migration

VARIABLE	RÔLE	CH.
<code>N8N_COMPRESSION_NODE_MAX_DECOMPRESSED_SIZE_BYTES</code>	Plafond de décompression, abaissé à 256 Mio en 3.0.	23
<code>N8N_COMPRESSION_NODE_MAX_ZIP_ENTRIES</code>	Nombre d'entrées d'archive, abaissé à 1 000 en 3.0.	23

Les valeurs par défaut changent d'une version à l'autre. La liste complète est maintenue dans la documentation officielle et doit être consultée à la version employée.

ANNEXE C

Grille de choix d'outil

RÉFÉRENCE

La grille du chapitre 2, ramenée à une page. Elle se lit de haut en bas : la première ligne dont la description correspond donne la famille d'outils, et le produit se choisit ensuite.

SI LE BESOIN EST...	ALORS LA FAMILLE EST
Un processus qui dure des jours ou des semaines, avec reprise exacte après panne	Orchestrateur d'exécution durable
Des millions d'enregistrements à transformer	Orchestrateur de données, ou une requête en base
Un processus humain à plusieurs niveaux, avec délégations et escalades sur délai	Moteur de processus métier
Un produit revendu ou intégré dans une offre commerciale	Accord de licence, ou outil sous licence permissive
Trois étapes entre deux applications courantes, sans exigence d'hébergement	Plateforme d'intégration grand public
Une intégration à logique riche, de durée courte, sur une infrastructure maîtrisée	Plateforme de workflow auto-hébergeable

Les questions à poser avant de trancher

- Combien de fois par mois, et quel est le coût d'une erreur passée inaperçue ?
- Qui hébergera, et qui assurera les mises à jour ?
- Le volume est-il facturé à la tâche, à l'opération, à l'exécution ou au serveur ?
- La licence autorise-t-elle ce que vous comptez en faire, y compris le revendre ?
- Que se passe-t-il si l'outil disparaît : les traitements sont-ils réécrivables ailleurs ?

La dernière question est la moins posée et la plus utile. Un parc d'automatisations dont la logique métier vit dans des sous-workflows nommés et documentés se transpose ; un parc dont la logique est enfouie dans des expressions et des nœuds de code ne se transpose pas.

ANNEXE D

Glossaire

RÉFÉRENCE

Les termes techniques employés dans ce livre sont conservés en anglais quand c'est l'usage professionnel. Ils sont glosés ici une fois pour toutes.

TERME	DÉFINITION
Agent	Boucle dans laquelle un modèle choisit d'appeler un outil ou de répondre, jusqu'à ce qu'il réponde.
Chaînage d'items	Lien conservé entre un item de sortie et l'item d'entrée dont il provient. Il permet de remonter aux données d'un nœud antérieur.
Clé naturelle	Valeur venue de l'émetteur qui identifie de façon stable un événement, et sur laquelle repose l'idempotence.
Exécution	Parcours complet d'un workflow, du déclenchement au terme. Unité de facturation sur l'offre hébergée.
Fenêtre de service	Période de vingt-quatre heures ouverte par un message entrant de client, pendant laquelle une réponse peut être envoyée sans gabarit approuvé.
Garde-fou	Contrôle placé avant ou après un modèle, qui bloque ou assainit ce qui ne satisfait pas une règle.
Idempotence	Propriété d'une opération dont la répétition avec les mêmes entrées produit le même état final qu'une exécution unique.
Item	Objet unitaire circulant entre deux nœuds, portant une clé de données structurées et éventuellement une clé de fichiers.
Itération implicite	Comportement par lequel un nœud recevant n items exécute son opération n fois, sans boucle explicite.
MCP	Protocole permettant à un assistant d'appeler des outils exposés par un serveur. $n8n$ peut tenir les deux rôles.
Mode queue	Architecture où l'instance principale orchestre et où des processus ouvriers exécutent, coordonnés par une file.
Nœud en grappe	Assemblage d'un nœud racine et de sous-nœuds qui étendent ses capacités : modèle, mémoire, outils, magasin vectoriel.
RAG	Procédé consistant à retrouver dans un corpus les passages pertinents pour une question, puis à les fournir au modèle avec la question.
Sortie d'erreur	Branche distincte vers laquelle sont dirigés les items dont le traitement a échoué.
Taux de couverture	Proportion des cas traités de bout en bout sans intervention humaine. N'atteint jamais cent pour cent.
Workflow d'erreur	

TERME	DÉFINITION
	Workflow déclenché par l'échec d'un autre, qui reçoit le détail de la panne et se charge de la signaler.

Fin de l'ouvrage. Les corrections, compléments de version et retours de terrain sont accueillis avec reconnaissance : un livre technique sur un outil qui publie une version par semaine ne reste juste que s'il est corrigé.

ANNEXE E

Notes et sources

RÉFÉRENCE

J'ai annoncé au début de cet ouvrage que je préférerais donner au lecteur de quoi vérifier plutôt que de quoi croire. Cette annexe rend cette promesse exécutable : elle dit sur quoi repose chaque partie, et ce qui aura bougé quand vous la lirez.

E.1 Trois niveaux, appliqués sans exception

Le comportement de l'outil ne s'appuie que sur la documentation officielle et les notes de version, avec mention de la version d'apparition quand elle est connue. Quand je décris ce que fait un nœud, cela se vérifie sur une page de documentation, pas sur un billet de blog.

Les tarifs et les grilles de prix sont donnés avec leur date de relevé et renvoyés vers l'éditeur concerné. Ils changent plusieurs fois par an, et j'ai choisi de décrire la structure des modèles économiques plutôt que de recopier des montants qui seront faux dans six mois.

Les retours de terrain sont présentés comme tels. Ils viennent de missions réelles, ils ne sont pas généralisables par eux-mêmes, et je ne les convertis jamais en règle universelle.

J'écarte par principe toute affirmation de performance ou d'économie provenant d'une source commerciale tierce. Je ne cite que des chiffres dont je peux expliquer le calcul, et je préfère montrer la méthode plutôt qu'un résultat que vous ne pourriez pas reproduire chez vous.

E.2 Sources primaires

- La documentation officielle de n8n, y compris les pages de déploiement, de mise à l'échelle, de sécurité et de référence des nœuds.
- Les notes de version et le journal des changements, qui donnent la version exacte d'apparition d'une fonctionnalité.
- Les pages de ruptures des versions 2.0 et 3.0, qui donnent les chemins de remplacement.

- Le texte de la Sustainable Use License et les précisions publiées par l'éditeur sur son périmètre.
- Les communications officielles de l'éditeur sur son actionnariat et ses partenariats.
- La documentation développeur de la plateforme WhatsApp Business et les annonces de tarification de son éditeur.

E.3 Notes par partie

PARTIE	CE SUR QUOI REPOSENT LES AFFIRMATIONS
I	Les critères d'éligibilité et la formule de gain net ne sont pas des données mesurées : ce sont des cadres de décision. Les faits vérifiables portent sur la licence, l'actionnariat de l'éditeur et les licences des alternatives citées.
II	Modèle de données, ordre d'exécution, modes du nœud de fusion, comportement du chaînage d'items, vérification native de signature sur les nœuds déclencheurs, réglages de nouvelle tentative et de déduplication : tout provient de la documentation et des notes de version.
III	Mode queue, concurrence, clé de chiffrement partagée, modes de stockage binaire, plafond de relais des réponses de webhook, télémétrie, coffres externes, contrôle de source : documentation officielle, avec les versions d'apparition.
IV	Nœuds conversationnels, agent et outils, modes des magasins vectoriels, garde-fous, évaluations : documentation officielle. L'économie du canal WhatsApp provient de la documentation développeur et des annonces de son éditeur.
V	Les six chapitres combinent des mécanismes documentés et des retours de terrain. Les mécanismes se vérifient ; les contraintes de contexte, les ordres de grandeur de durée et les pièges rencontrés relèvent de l'expérience et sont signalés comme tels.
VI	La liste des éléments retirés en version 3.0 et leurs remplacements proviennent de la page officielle des ruptures, qui était encore en cours de complément à la date de rédaction.

E.4 Ce qui aura bougé quand vous lirez

Trois zones de ce livre vieilliront plus vite que les autres, et il vaut mieux le dire que de le laisser découvrir.

La version de la plateforme. J'ai écrit sur la 2.38 ; la 3.0 était annoncée pour octobre 2026. Le chapitre 23 traite cette rupture, mais la page officielle des ruptures continue d'être complétée à l'approche de la sortie. Relisez-la le jour de votre migration.

La tarification du canal WhatsApp. Elle change au 1^{er} octobre 2026, et elle avait déjà changé deux fois dans les dix-huit mois précédents. Le chapitre 12 donne la structure du modèle et la méthode pour mesurer votre coût réel depuis vos propres exécutions ; il ne donne aucune grille tarifaire, volontairement.

Les capacités des modèles. Toute la partie IV décrit des mécanismes de la plateforme, pas des performances de modèles. C'est délibéré : les mécanismes durent, les classements de modèles ne durent pas trois mois.

E.5 Corrections

Un livre technique sur un outil qui publie une version par semaine ne reste juste que s'il est corrigé. Les erreurs constatées, les comportements qui auraient changé et les retours de terrain qui contredisent ce que j'avance sont accueillis avec reconnaissance, et intégrés à la révision suivante.

Je tiens particulièrement aux contre-exemples. Une automatisation qui a échoué pour une raison que ce livre n'anticipe pas vaut davantage, pour la prochaine édition, que dix confirmations de ce qu'il affirme déjà.